REPUBLIQUE DU SENEGAL

UN PEUPLE – UN BUT - UNE FOI

Ministère de la Formation Professionnelle, de l'Apprentissage et de l'Artisanat (MFPAA)

ECOLE DU CODE SONATEL ACADEMY (ECSA)

Ecole Police, Immeuble Scalène Mermoz lot B, Rue KA 05, Dakar

Mémoire de fin de cycle pour l'obtention du certificat de spécialisation

Spécialité : Développement Web et Mobile

Sujet : Blacklistage des numéros à répétition d'appels vers les numéros du service client SONATEL

Présenté par :	Maitre de Stage :	Encadré par :
Oumar SY	M. Djiby Thiaw	M. Birane Baila Wane
	Ingénieur Développeur/Intégrateur Lead Chapter CDM	Ingénieur Informaticien Coach ODC

Année académie 2024-2025

Dédicace

Je dédie ce mémoire :

À **mes parents**, pour leur amour, leur soutien inestimable et les sacrifices qu'ils ont consentis pour ma réussite.

À **ma famille**, pour leur présence constante, leurs encouragements et leur confiance en moi.

À **ma jumelle Fatima Diouf**, ma plus que sœur qui est toujours à mes côtés.

À **mes amis**, pour leur soutien moral et leurs conseils tout au long de ce parcours.

À **mes camarades de promotion à la Sonatel Académie**, pour les moments de partage, de collaboration et de dépassement de soi vécus ensemble. Et à toutes celles et ceux qui, de près ou de loin, ont contribué à la réalisation de ce travail.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet de stage et à l'élaboration de ce rapport. Leur soutien, leurs conseils et leur expertise ont été essentiels pour mener à bien cette expérience enrichissante.

Tout d'abord, je souhaite adresser mes sincères remerciements à SONATEL Academy, et plus particulièrement à mon coach Aly Niang, pour son encadrement rigoureux et ses précieux conseils tout au long de ma formation à l. Sa disponibilité et son professionnalisme ont été une source d'inspiration et un levier clé pour la réussite de ce projet. Je remercie également Birane Wane, mon encadreur académique, pour son accompagnement méthodologique et ses retours constructifs qui ont permis d'améliorer la qualité de ce travail.

Je suis particulièrement reconnaissant envers mon maître de stage, Djiby Thiaw, pour sa confiance, son expertise et son soutien constant. Ses orientations stratégiques et ses précieux conseils m'ont permis de mieux appréhender les enjeux du projet et de le mener à bien. J'exprime également toute ma gratitude à Abdoulaye Seye, chef de département du service One Team, pour son accueil chaleureux et son leadership exemplaire.

Mes remerciements s'adressent également aux membres du service One Team qui ont contribué à la réussite de ce projet : Malick Coly, Mouhamed Mbengue, Naby Dia, Fatou Sall, Ibnou Abass, Ndeye Fatou Lagnane, Samir.... Merci pour votre collaboration, vos idées constructives et votre disponibilité. Vous avez joué un rôle essentiel dans la réalisation de ce projet.

Je tiens également à remercier ma collègue stagiaire **Dieynaba Ndiaye**, pour sa gentillesse, son esprit social et son soutien quotidien. Travailler à ses côtés a été une expérience agréable et enrichissante.

Dans un cadre plus personnel, je souhaite remercier mes amis, en particulier **Fatoumata Diarietou Bah**, pour sa complicité et son soutien moral. Je n'oublie pas ma promotion, qui m'a accompagné tout au long de ce parcours académique et professionnel.

Enfin, je réserve une pensée spéciale à mes parents, qui ont toujours cru en moi et m'ont soutenu dans mes choix. Leur amour, leur patience et leurs encouragements ont été un moteur constant pour aller de l'avant et donner le meilleur de moi-même.

Ce projet et ce stage sont le fruit d'un travail collectif, et je mesure pleinement la chance d'avoir pu compter sur autant de personnes bienveillantes et compétentes. Qu'ils trouvent ici l'expression de ma sincère reconnaissance. Merci infiniment à tous !

Table des Figures

Figure 1: marché et concurrence 3
Figure 2: Equipe et cadre de travail 4
Figure 3: présentation Jhipster 8
Figure 4: exemple schema JDL 9
Figure 5: jenkins de notre application 13
Figure 6: Phases du cycle en « cascade » 16
Figure 7: Phases du cycle en « V ». 17
Figure 8: Représentation d'un développement itératif et incrémental 19
Figure 9: Le cycle de vie Scrum 23
Figure 10: Le cycle de vie eXtreme programming (XP) (Valentine, 2011) 26
Figure 11: acteur 30
Figure 12: Diagrammes de cas d'utilisation de notre application 31
Figure 13: Diagramme de classe de l'application 39
Figure 14: Diagramme de séquence d'authentification 40
Figure 15: Diagramme de séquence blacklister un numéro 41
Figure 16: Diagramme de séquence blacklister un numéro 42
Figure 17: Figma 43
Figure 18: Prototype parametre 44
Figure 19: Prototype blacklistage 45
Figure 20: Prototype de l'historique 46
Figure 21: Présentation de GDD 48
Figure 22: La page de connexion 50
Figure 23: Backend Paramètre 51
Figure 24: Composant Paramètre 52
Figure 25: html de paramètre 53
Figure 26: paramètre 53
Figure 27: html blackliste 55
Figure 28: page Blacklistage 56
Figure 29: Backend historique 57
Figure 30: le composant Historique 57
Figure 31: html d'Historique 58
Figure 32: la page historique 58
Figure 33: notre Dashboard 59

Table des Tableaux

Tableau 1: comparaison des frameworks 13
Tableau 2: Comparaison des méthodes Scrum et eXtreme Programming (XP) 26
Tableau 3: Comparaison entre approche classique et approche agile 28
Tableau 4: Use-case "se connecter" 32
Tableau 5: Use-case "blacklister un numéro" 32
Tableau 6: Use-case " Importation en masse” 33
Tableau 7: Use-case "Télécharger le modèle” 33
Tableau 8: Use-case "Lister numéros blacklistés” 34
Tableau 9: Use-case "Ajouter Parametre" 34
Tableau 10: Use-case " Lister paramètres” 35
Tableau 11: Use-case « Whitelister la sélection » 36
Tableau 12: Use-case "Exporter" 36
Tableau 13: Use-case " Lister l'historique" 37
Tableau 14: Comparaison GDD vsTrello 48

Résumé

Dans un contexte marqué par l'application du décret relatif à la gratuité des appels vers le service client SONATEL, les plateaux ont été confrontés à une augmentation significative des appels plaisantins et répétitifs. Ces appels nuisent à la qualité de service en saturant les accès et en empêchant les clients légitimes d'être pris en charge.

Pour y remédier, ce projet vise à automatiser la gestion des numéros clients via des mécanismes de « Blacklistage » et de « Whitelistage », tant unitairement que par lots. Une solution technologique a été conçue et développée pour paramétrer ces processus sur la base de critères spécifiques (nombre d'appels autorisés, période d'observation, durée de blocage, etc.). Ce rapport détaille les étapes de conception, réalisation et déploiement de cette solution.

Abstract

Following the implementation of the decree mandating free calls to SONATEL's customer service, there has been a significant increase in prank and repetitive calls. These calls have negatively impacted service quality by overloading the system and preventing legitimate customers from being properly assisted. To address this issue, the project aimed to automate the management of customer phone numbers through blacklist and whitelist mechanisms, both individually and in batches. A technological solution was designed and developed to configure these processes based on specific criteria such as the number of allowed calls, observation period, and blocking duration. This report outlines the design, development, and deployment phases of the solution, which seeks to optimize call flow management and ensure better access to customer service.

Glossaire

ARTP : Autorité de Régulation des Télécommunications et des Postes

CDM : Centre de Développements Mutualisés

DSI : Direction des Systèmes d'Information

JDL: JHipster Domain Language

IAM: Identity and Access Management

OPR : Observabilité et Pilotage du Run

PMO/RSM: Project Management Officer/Relationship Management

SSO : Single Sign-On

SONATEL : Société Nationale des Télécommunications du Sénégal

UML : Langage de Modélisation Unifié

Sommaire

INTRODUCTION..... 1

I. Présentation de l'organisme d'accueil.....3

II. Présentation du sujet et des objectifs.....5

III. La méthodologie.....17

IV. Conception détaillées.....32

V. Le développement de l'application.....49

CONCLUSION GÉNÉRALE.....56

Webographie.....57

INTRODUCTION

Les services clients des opérateurs de télécommunications constituent un maillon essentiel dans la relation avec la clientèle. Le groupe Sonatel, en tant que leader des télécommunications au Sénégal et dans plusieurs autres pays d'Afrique de l'Ouest, accorde une importance capitale à la qualité de ses services d'assistance à la clientèle.

La mise en œuvre récente du décret relatif à la gratuité des appels vers le service client a engendré des changements significatifs dans les habitudes de consommation. Si cette mesure vise à faciliter l'accès au service client pour les utilisateurs, elle a également généré des effets secondaires non anticipés. En effet, on observe une multiplication des appels vers les plateaux de service client, dont une proportion non négligeable correspond à des appels plaisantins ou répétitifs sans motif légitime.

Les analyses effectuées sur les données des plateaux révèlent que certains numéros contactent le service client plus de 10 fois par jour. Cette situation a pour conséquence directe une saturation des lignes d'accès aux plateaux, compromettant ainsi la prise en charge des clients qui appellent pour de réels besoins d'assistance ou d'information.

Face à cette problématique, Sonatel a identifié la nécessité de mettre en place un système automatisé capable de détecter et de gérer les numéros en répétition d'appels, afin d'optimiser les ressources du service client et d'améliorer la qualité de service offerte aux clients légitimes.

Tout d’abord, je décrirai le Groupe SONATEL et son secteur en insistant sur l’enjeu de ce projet. Puis je présenterai mes missions lors de ce stage avant de dresser un bilan de celui-ci. A travers ce mémoire, je détaillerai mes 3 mois de stage, en commençant par une présentation de l’organisme d’accueil ; ensuite, j’enchaînerai par une présentation du sujet et de l’objectif du stage. Et enfin, la dernière partie portera sur l’ensemble du travail effectué, autrement dit, j’aborderai la méthode, les différents résultats que j’ai pu obtenir, l’analyse UML et ensuite, je pourrai éventuellement vous parler des perspectives envisageables. Puis, je dresserai un bilan, autant au niveau des connaissances utilisées qu’au niveau de celles qui m’ont été apportées par l’entreprise.

I. Présentation de l'organisme d'accueil

I.1. Présentation de la société

Le Groupe Sonatel offre des solutions globales de télécommunications dans les domaines du fixe, du mobile, de l'Internet, de la télévision et des données au service des particuliers et des entreprises. Leader au Sénégal, présente au Mali depuis 2002, en Guinée Conakry et Guinée Bissau depuis 2007, en Sierra Léone en 2016, le groupe Sonatel est l'opérateur global et sous régional de référence.

Sonatel a construit un réseau moderne, entièrement numérisé par des boucles de transmission et des liaisons internationales par câbles sous-marins à fibres optiques haut débit. Sonatel dispose de l'une des bandes passantes Internet les plus importantes d'Afrique (6.5 Gbits/s).

Le groupe Sonatel est une entreprise structurante pour l'économie de ses pays de présence et a fait du Sénégal, son pays d'origine, un hub de trafic et un acteur majeur dans le développement des télécommunications en Afrique et dans le monde.
A travers sa marque commerciale Orange, le groupe Sonatel fait vivre à ses millions de consommateurs les meilleures opportunités de la convergence entre les technologies du mobile, de l'internet et de la télévision.

Avec plus de 41,1 millions de clients dans tout le groupe et un chiffre d'affaires annuel de plus de 1 200 milliards de francs CFA, SONATEL occupe une position stratégique dans le paysage économique national et sous-régional. L'entreprise emploie plus de 3 000 collaborateurs et dispose d'un réseau de distribution comprenant des agences physiques, des franchises et des points de vente partenaires à travers tout le pays.

Le service client constitue une fonction cruciale pour SONATEL, avec plusieurs plateformes téléphoniques réparties sur le territoire national, notamment à Dakar, Thiès et Saint-Louis. Ces centres d'appels traitent quotidiennement plusieurs dizaines de milliers d'appels concernant des demandes d'information, des réclamations, des souscriptions aux services, et du support technique.

Le stage s'est déroulé au sein de la Direction des Systèmes d'Information plus précisément dans le département Fusion Team, qui supervise les activités des centres d'appels et veille à l'amélioration continue de la satisfaction client.

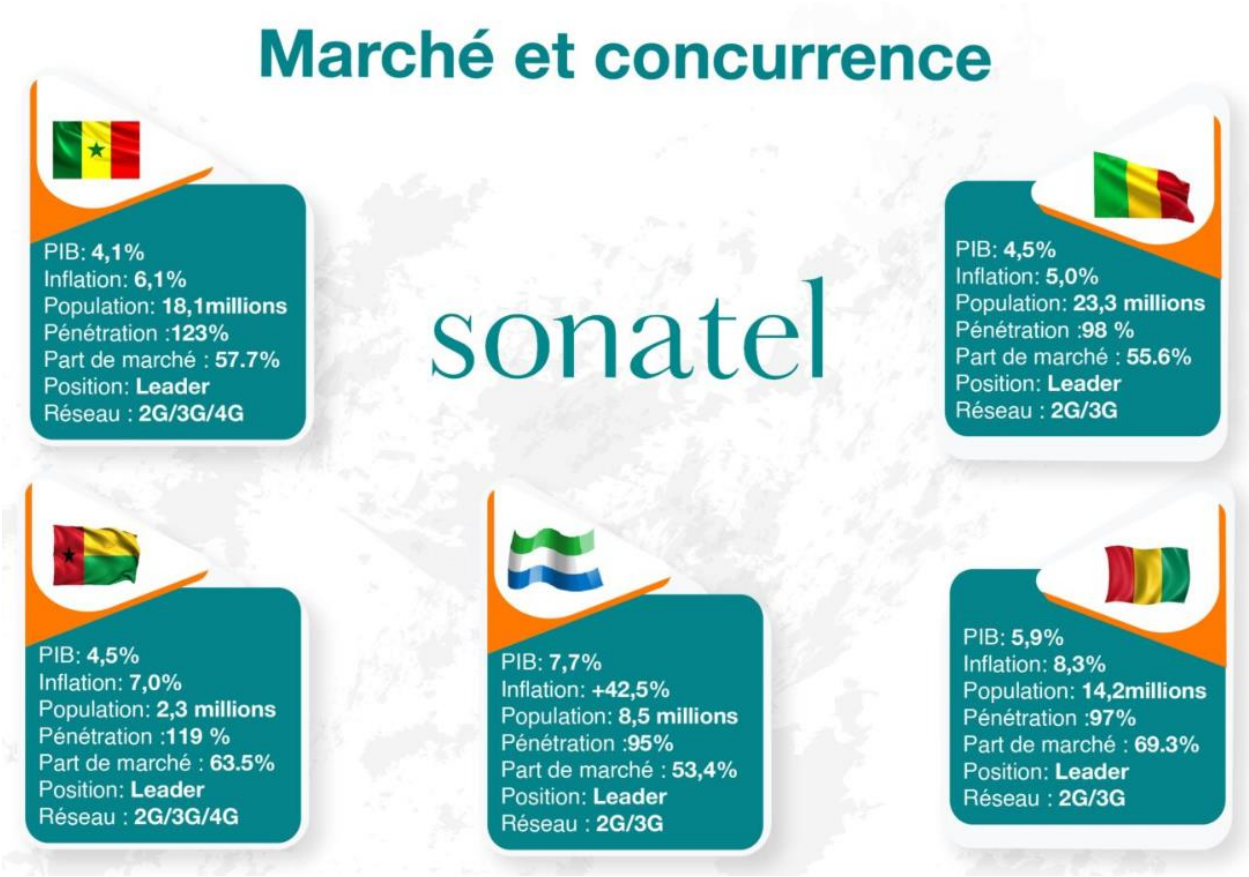

Figure 1: marché et concurrence

https://sonatel.sn/groupe-sonatel/presence/

I.2. Présentation de l'équipe et du cadre de travail

Durant notre immersion, nous avons été affectés au département Fusion Team SI Relation Client (**FT_SIRC**), qui relève de la Direction du Système d'Information (**DSI**). Cette équipe se concentre sur deux sous-domaines principaux : le Call Center/Digital Content et le Customer Management. Dans ces domaines, elle est responsable de la gestion de plusieurs produits essentiels pour l'interaction avec les clients et le support numérique. La FT_SIRC est divisée en trois chapter. Le premier, le CDM (Centre de Développements Mutualisés), est chargé du développement des produits de l'équipe, en veillant à ce qu'ils répondent aux besoins des clients. Le deuxième chapter, l'IPR (Innovation et Projets), se concentre sur la gestion des projets d'innovation.

Enfin, le chapter OPR (Observabilité et Pilotage du Run) assure la gestion opérationnelle des produits en veillant à leur bon fonctionnement, leur performance et leur conformité aux attentes des utilisateurs finaux. En outre, la FT_SIRC dispose d'un PMO/RSM (Project Management Officer/Relationship Management) qui coordonne les projets et les équipes en utilisant des

méthodologies agiles. Cette structure est directement rattachée à la DSI de la Fusion Team SI Relation Client est le suivant :

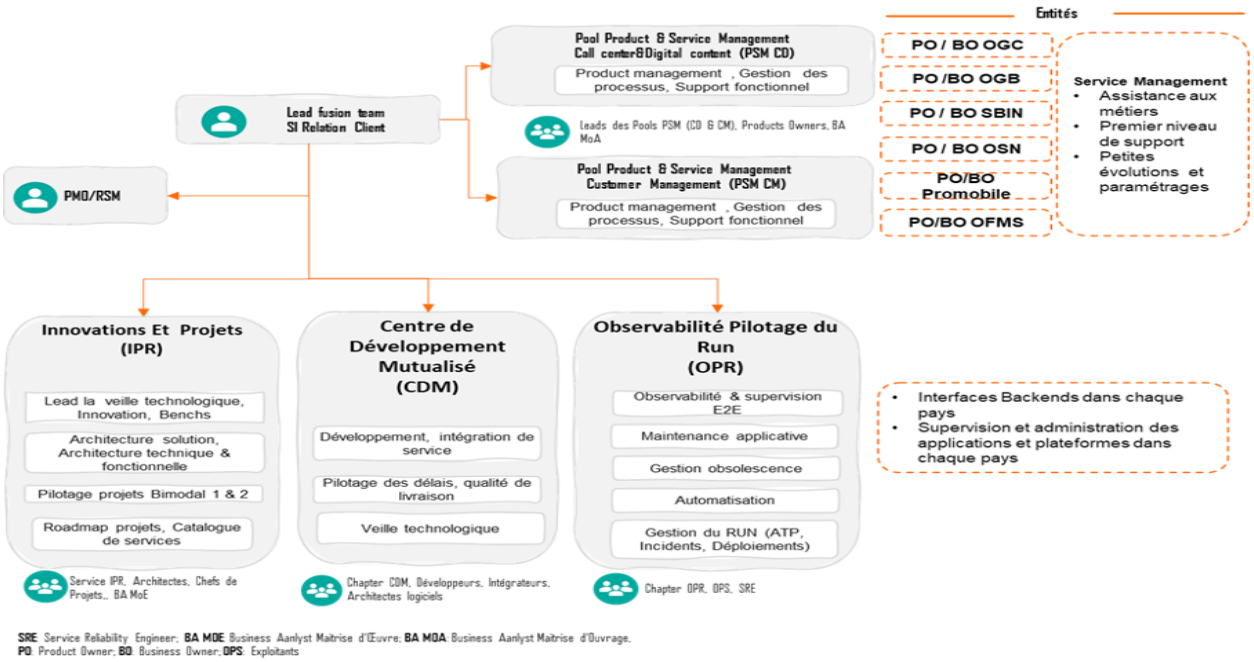

Figure 2: Equipe et cadre de travail

II. Présentation du sujet et des objectifs

1. Présentation du Sujet

Le projet vise à développer une solution blacklistage des appels répétitifs permettant d'identifier et de gérer efficacement les appelants effectuant des appels répétitifs vers le service client de SONATEL, tout en respectant le cadre légal de la gratuité des appels instaurée par le décret gouvernemental.

Cette solution a pour objectif principal de filtrer les appels non productifs qui saturent actuellement les plateaux téléphoniques, afin de garantir une meilleure accessibilité aux clients ayant des besoins légitimes. La solution proposée s'articule autour de trois axes principaux :

Système d'identification des appels répétitifs : Un mécanisme automatisé permettant de détecter en temps réel les numéros effectuant des appels multiples dans un intervalle de temps défini.

Parcours client différencié : Un système de routage intelligent qui, en fonction du profil d'appel identifié, oriente l'appelant vers un traitement adapté.

Plateforme analytique : Un outil d'analyse permettant de suivre l'évolution des comportements d'appel, d'évaluer l'efficacité des mesures mises en place et d'affiner les règles de détection.

Le projet s'inscrit dans une démarche d'amélioration continue de la qualité de service, tout en optimisant l'utilisation des ressources humaines et techniques de SONATEL.

2. Contexte

La mise en œuvre du décret relatif à la gratuité des appels vers le service client génère une multiplication des appels vers les plateaux Service Client.L'effet de bord de l'application de ce décret est la pollution des plateaux avec la réception d'appels plaisantins à répétition. Les analyses faites sur les données des plateaux indiquent qu'il y a des numéros qui appellent plus de 10 fois par jour. Ce qui a pour impact, une saturation des accès aux plateaux provocant ainsi une non prise en charge des clients qui appellent pour de réels besoins.

3. La problématique de l'existant

Depuis que le gouvernement sénégalais a promulgué un décret rendant obligatoire la gratuité des appels vers les services clients des opérateurs de télécommunications. Cette mesure, destinée à renforcer la protection des consommateurs et à améliorer l'accessibilité des services, a eu des répercussions importantes sur le fonctionnement des plateaux téléphoniques de SONATEL.

Si l'intention du législateur était louable, la mise en application de ce décret a rapidement engendré des effets secondaires non anticipés. Les analyses des données d'appels réalisées par le département Qualité et Performance ont révélé une augmentation spectaculaire du volume d'appels entrants, avec une hausse de plus de 70% dans les trois mois suivant l'entrée en vigueur du décret.

Plus préoccupant encore, ces analyses ont mis en évidence un phénomène nouveau : l'émergence d'appels dits "non productifs" ou "plaisantins", caractérisés par leur répétitivité et leur absence de motif légitime. Les données collectées montrent que certains numéros appellent plus de 10 fois par jour le service client, parfois jusqu'à 30 ou 40 fois pour les cas les plus extrêmes.

Cette situation a généré plusieurs problèmes opérationnels majeurs :

- Saturation des lignes d'accès aux plateaux téléphoniques
- Augmentation significative du temps d'attente pour les clients ayant des besoins réels
- Baisse des indicateurs de performance (taux de décroché, temps moyen de traitement)
- Détérioration de l'expérience client
- Stress accru pour les agents du service client confrontés à des appels abusifs ou sans objet

- Coûts supplémentaires liés a la nécessité d'augmenter les ressources humaines et techniques

Les méthodes traditionnelles de gestion des flux d'appels se sont révélées inadaptées face à ce phénomène nouveau, ce qui a conduit SONATEL à envisager une solution blacklistage des numéros à répétition d'appels capable d'identifier et de traiter spécifiquement les appels répétitifs tout en respectant le cadre légal de la gratuité des appels.

Il existe un formulaire dans la plateforme OMEGA qui permet de charger un fichier de numéros d'appels comportant 2 champs : MSISDN, la valeur « true » en vue Blaclistage. Insérer un numéro dans cette base a pour conséquences d'interdire tout appel de ces numéros vers les numéros de service client.

Cette base est actuellement utilisée et mise à jour périodiquement. Depuis la mise en place de la gratuité sur certains numéros du service client, les clients en répétition d'appels d'ordre 4 sont extraits tous les jours par DPPS des reportings d'appels et transmis à DSC/DIS en vue chargement dans cette base.

4. Points faibles du (des) processus actuels

La collecte des numéros en répétition d'appels est manuelle, chronophage et est sources d'erreurs. L'intégration de nouveaux numéros dans la Blacklist se fait de 2 manières :

- par export de la base + ajout des nouveaux numéros à blacklister et chargement de la liste nouvellement constituée dans la base. Elle vient écraser la base existante//enlever les doublons
- par ajout unitaire de numéros dans la base (une interface de chargement unitaire de numéros disponible) Le retrait de numéros de la base se fait selon le même procédé.
- Export de la base et suppression des numéros puis injection du fichier résultant qui vient écraser la base
- Par retrait unitaire de numéros Un numéro blacklisté est blacklisté sur l'ensemble des numéros du service client. Il n'est pas possible de limiter les interdictions d'appels vers des plateaux spécifiques (loi du tout ou rien). Si un numéro est dans la Blacklist, il ne peut appeler aucun numéro du service client Il n'est pas possible de charger un fichier dont la taille dépasse 1000 enregistrements sans des risques de dégradation des config system, pouvant entrainer une perte totale de service.

5. Objectifs

L'objectif principal de ce projet est de concevoir et développer une solution de blacklistage des appels répétitifs vers le service client de SONATEL, permettant de réduire l'impact des appels plaisantins et répétitifs sur la qualité de service. Cette solution repose sur des mécanismes de « Blacklistage » et de « Whitelistage », paramétrés selon des critères

spécifiques tels que le nombre d'appels autorisés, la période d'observation et la durée de blocage, afin d'optimiser les ressources des plateaux téléphoniques et d'améliorer l'accessibilité aux clients légitimes.

6. Les outils de travail et les langages utilisés

6.1. Les outils de travail

IntelliJ IDEA : Environnement de développement intégré pour Java
Git : Système de contrôle de version
Maven : Outil de gestion de dépendances et de build
Postman : Pour tester les API REST

BitBucket : est un service web d'hébergement et de gestion de développement logiciel utilisant le logiciel de gestion de versions Git. Pour travailler avec un dépôt distant nous utilisons bithBacket. Ce dernier est un outil de communication et de collaboration entre plusieurs développeurs (ou tout autre personne qui écrit du texte). C'est une interface web créée pour faciliter l'interaction avec Git

Pour la réalisation de notre solution de blacklistage des numéros à répétiotion d'appels vers les numéros du service client SONATEL, nous avons opté pour une architecture moderne basée sur les microservices. Cette approche facilite le développement, le déploiement et la maintenance de services indépendants qui communiquent entre eux via des API REST.

JHipster : Générateur d'applications

Présentation de JHipster

JHipster (Java Hipster) est un générateur d'applications open source qui permet de créer rapidement des applications modernes avec un backend Java/Spring Boot et un frontend Angular, React ou Vue.js. Pour notre projet, nous avons particulièrement exploité sa capacité à générer une architecture microservices.

Le principe de **jHipster** est relativement simple. Vous définissez un **schéma d'entités** (leurs définitions + leurs relations) et à partir de ce schéma, jHipster va pouvoir générer :

- La création des entités dans la base de données du projet
- La définition des entités dans le back-end
- La mise en place d'une API et des requêtes CRUD pour gérer les entités
- La modélisation des entités dans le front-end
- Les services d'accès aux entités côté front-end

- Une partie d'administration des entités dans le projet front
- La gestion des comptes utilisateurs et de leurs rôles

```
INFO! Using JHipster version installed globally
INFO! Executing jhipster:app

                          https://www.jhipster.tech

Welcome to JHipster v6.10.3
Application files will be generated in folder: /home/bear/jhipsterDemoApp

_____________________________________________________________________________________________________
________

  Documentation for creating an application is at https://www.jhipster.tech/creating-an-app/
  If you find JHipster useful, consider sponsoring the project at https://opencollective.com/generator-
jhipster

_____________________________________________________________________________________________________
________

? May JHipster anonymously report usage statistics to improve the tool over time? No
? Which *type* of application would you like to create? (Use arrow keys)
❯ Monolithic application (recommended for simple projects)
  Microservice application
  Microservice gateway
  JHipster UAA server
```

Figure 3: présentation Jhipster

JDL Studio

JDL (JHipster Domain Language) Studio est un outil en ligne essentiel dans notre processus de développement avec JHipster. Cet éditeur visuel nous a permis de :

- **Modéliser graphiquement** les entités de notre domaine métier
- **Définir les relations** entre les différentes entités (one-to-many, many-to-many, etc.)
- **Spécifier les attributs** de chaque entité avec leurs types et contraintes
- **Générer automatiquement** le code JDL correspondant au modèle créé

Le JDL est un langage spécifique à JHipster qui permet de décrire toutes les entités et leurs relations dans un format compact et lisible.

```
entity Historique{
  date ZonedDateTime
  action Action
  parService String
   msisdn  String
 }
 entity HistoriqueConnexion{
   action String
   date ZonedDateTime
   nom String
   prenon String
   login String
   email String
   role String
 }

 entity Dashboard{
 libelle String required unique
 url TextBlob required
 roles TextBlob required

 }
enum Action {
   BLACKLISTER , WHITELISTER
}
relationship ManyToOne {
  Historique{createdBy(login)} to User with builtInEntity
}

dto * with mapstruct
service * with serviceImpl
filter *
paginate * with pagination
```

Figure 4: exemple schema JDL

Une fois ce modèle défini dans JDL Studio, nous avons exporté le code JDL et l'avons utilisé pour générer automatiquement les entités, repositories, services et contrôleurs correspondants dans nos microservices.

JHipster Microservice

JHipster Microservice nous a permis de générer des services indépendants, chacun avec sa propre base de données, sa propre logique métier et son propre domaine fonctionnel. Les avantages de cette approche incluent :

Scalabilité : Possibilité d'adapter les ressources selon la charge de chaque service

Résilience : La défaillance d'un service n'affecte pas l'ensemble du système

Facilité de développement : Chaque microservice peut être développé, testé et déployé indépendamment

Les principaux microservices développés dans notre solution comprennent :

Service de gestion des appels dans Omega,

Service de gestion des historiques

JHipster Gateway

La passerelle (Gateway) JHipster joue un rôle central dans notre architecture. Elle sert de point d'entrée unique pour toutes les requêtes clients et offre les fonctionnalités suivantes :

- **Routage des requêtes** vers les microservices appropriés
- **Authentification et sécurité** centralisées
- **Interface utilisateur unifiée** pour l'ensemble de l'application
- **Load balancing** pour distribuer la charge entre les instances de microservices

5.2. Langages de programmation

1. Backend

Pour le développement du backend, nous avons utilisé :

Java : Langage principal pour le développement des microservices

Spring Boot : Framework pour le développement d'applications Java

Spring Cloud : Ensemble d'outils pour la gestion des microservices

Hibernate/JPA : Pour la persistance des données et l'ORM

2. Frontend

Pour le développement de l'interface utilisateur, nous avons exploité :

Angular : Framework JavaScript pour le développement d'applications web dynamiques

TypeScript : Langage de programmation qui ajoute le typage statique à JavaScript

HTML5/CSS3 : Pour la structure et la présentation de l'interface

Bootstrap (Boosted): Framework CSS pour le design responsive

3. Bases de données

Notre solution utilise deux systèmes de gestion de bases de données différents selon les besoins spécifiques des microservices :

a. Oracle Database

Utilisation : Pour les données critiques et les services nécessitant des fonctionnalités avancées

Avantages exploités :

Performances élevées pour les requêtes complexes

Fiabilité et robustesse pour les données critiques

Fonctionnalités avancées de sécurité
Compatibilité avec l'infrastructure existante de SONATE

b. MySQL

Utilisation : Pour les microservices nécessitant moins de ressources ou pour les données moins critiques

Avantages exploités :
Légèreté et facilité de déploiement
Performances adaptées aux besoins des microservices secondaires
Coût réduit par rapport à Oracle
Large communauté et documentation abondante

5.3. Outils déploiement

1. Conteneurisation et orchestration

Docker : Utilisé pour la conteneurisation des microservices

Docker Compose : Déployé pour la gestion des environnements de développement

2. Kubernetes et OpenShift

Notre architecture de microservices a été déployée sur une plateforme d'orchestration de conteneurs robuste combinant Kubernetes et OpenShift :
Kubernetes :

Auto-scaling : Adaptation automatique du nombre d'instances selon la charge
Service discovery : Découverte dynamique des services
Load balancing : Répartition intelligente du trafic entre instances
Self-healing : Redémarrage automatique des conteneurs défaillants
Déploiement progressif : Stratégies de rolling updates pour des mises à jour sans interruption
OpenShift (Distribution Red Hat de Kubernetes) :
Interface utilisateur web : Console d'administration simplifiée
Gestion intégrée des images : Registry interne pour les images de conteneurs
CI/CD natif : Intégration de pipelines de déploiement continu
Sécurité renforcée : Isolation des conteneurs et politiques de sécurité avancées
Support multi-tenant : Séparation logique des environnements et des projets
L'utilisation combinée de Kubernetes et OpenShift nous a permis d'obtenir une plateforme hautement disponible, résiliente et évolutive pour notre solution de gestion des clients en répétition d'appels SONATEL.

1. Intégration et déploiement continus (CI/CD)

✓ **Jenkins**

Dans notre projet, **Jenkins** a joué un rôle fondamental en tant que serveur d'intégration continue, nous permettant d'automatiser l'ensemble du processus de build, test et déploiement :

Architecture et configuration :

- **Pipeline as Code** : Définition des workflows d'intégration continue dans des fichiers Jenkinsfile
- **Architecture maître-agents** : Distribution des builds sur plusieurs nœuds pour optimiser les performances
- **Intégration avec Git** : Déclenchement automatique des builds à chaque push sur les branches principales

✓ **SonarQube**

SonarQube a été intégré comme outil central d'analyse statique du code et de qualité continue :

Fonctionnalités exploitées :

- **Analyse de la qualité du code** : Détection des bugs, vulnérabilités et code smells
- **Mesure de la couverture de tests** : Suivi du pourcentage de code couvert par les tests unitaires
- **Détection de la dette technique** : Quantification et visualisation de la dette technique
- **Quality Gates** : Définition de critères de qualité bloquants pour les déploiements

L'utilisation de SonarQube a permis d'établir une culture de qualité au sein de l'équipe de développement, assurant la livraison d'un code maintenable, sécurisé et performant pour notre solution de gestion des clients en répétition d'appels vers les numéros du service client SONATEL.

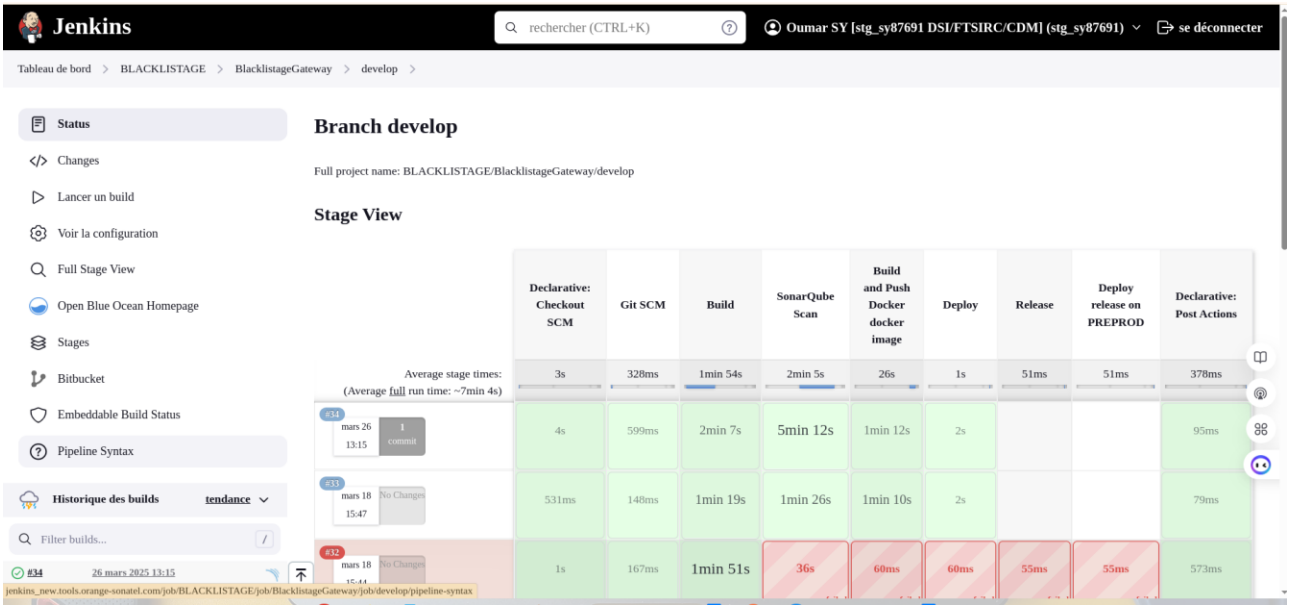

Figure 5: jenkins de notre application

5.4 Étude comparative des frameworks existants

a. Comparaison des frameworks pour microservices

Tableau 1: comparaison des frameworks

Framework	Avantages	Inconvénients	Pertinence pour le projet
JHipster	Génération rapide, complète et cohérente d'applications	Courbe d'apprentissage initiale	Très élevée (choisi)
Spring Boot seul	Légèreté, flexibilité	Configuration manuelle plus importante	Élevée
Quarkus	Performance, empreinte mémoire réduite	Écosystème moins mature	Moyenne
Micronaut	Démarrage rapide, faible consommation mémoire	Communauté plus restreinte	Moyenne

b. Justification du choix de JHipster

Notre choix s'est porté sur JHipster pour les raisons suivantes :

Productivité accrue : Génération automatique du code boilerplate

Architecture cohérente : Utilisation des meilleures pratiques pour les microservices

Intégration native avec Spring Boot et Angular

Support étendu pour les bases de données Oracle et MySQL

Documentation complète et communauté active

5.5 Sécurité et authentification

Keycloak

Pour la gestion de l'authentification et des autorisations, nous avons implémenté **Keycloak**, une solution open source d'Identity and Access Management (IAM) :

- **Single Sign-On (SSO)** : Authentification unique pour l'ensemble des microservices
- **Gestion centralisée des utilisateurs** : Administration simplifiée des comptes et rôles
- **Support de protocoles standards** : OAuth 2.0, OpenID Connect et SAML 2.0
- **Intégration avec JHipster** : Configuration facilitée grâce au support natif

L'intégration de Keycloak dans notre architecture microservice a permis de :

- Centraliser la gestion des identités
- Définir des politiques d'accès granulaires
- Sécuriser l'accès aux API exposées par chaque microservice
- Établir des sessions utilisateur sécurisées avec JWT (JSON Web Tokens)

La mise en place de Keycloak a nécessité la configuration de realms, clients, et rôles spécifiques adaptés aux besoins de sécurité de notre solution de gestion des clients en répétition d'appels.

III. La méthodologie

Dans le cycle de vie de notre projet, la conception représente une phase primordiale et déterminante pour produire une application de haute qualité. C'est dans ce stade que nous devons clarifier en premier lieu la vue globale, en décrivant l'architecture générale que nous allons suivre dans la partie réalisation de notre projet. Puis, dans un deuxième lieu nous allons détailler notre choix conceptuel à travers plusieurs types de diagrammes. Dans la suite de ce mémoire, nous ferons une distinction nette entre les méthodes de gestion de projet classiques et les méthodes agiles. Les méthodes de gestion de projet classiques se basent sur une approche dite « prédictive » ou « déterministe ». Toutes les étapes du projet doivent être planifiées dès son lancement et strictement respectées.

Les méthodes classiques reposent sur le fait que les étapes du projet se suivent de façon séquentielle et qu'une étape ne peut être entamée sans que la précédente ne soit terminée. Par ailleurs, une modification du projet en aval aura des conséquences potentiellement importantes en amont. C'est pour cela qu'il existe une grande rigidité dans l'adoption de ces méthodes car un changement inattendu dans une étape du projet peut avoir pour conséquence des pertes financières voire un échec du projet. Le développement informatique s'est basé depuis les années soixante-dix et se base encore sur ces méthodes classiques. Hors l'entreprise est confrontée à des demandes de clients qui évoluent fréquemment durant le projet. Suite à ce constat, des méthodes plus souples, dites « agiles », sont apparues au début des années 2000, afin d'adapter le projet à des demandes qui peuvent évoluer. Plutôt que d'essayer de prédire toutes les étapes du projet, le principe est de s'adapter à la demande avec une grande réactivité. Nous ne chercherons pas dans ce mémoire à balayer d'un revers de la main les méthodes classiques ; celles-ci proposent par exemple des pratiques qui sont reprises dans certaines méthodes agiles, comme les tests unitaires et fonctionnels. L'objet de cette section est d'apprécier avec objectivité les apports que peuvent fournir les méthodes agiles en les comparant notamment avec celles dites classiques.

1. Les méthodes classiques :

A. Fondements théoriques

Les méthodes classiques existent depuis plusieurs décennies. En effet, le cycle de vie dit « en cascade » a été formalisé par Royce (1970), un informaticien américain. Les méthodes classiques ont été conçues pour apporter un cadre formel et structuré dans la gestion de projet.

Les principales méthodes

a. Le cycle « en cascade »

Le cycle en cascade repose sur les principes suivants :

- Le cycle de vie du projet est constitué de phases séquentielles.

- On ne peut passer à la phase suivante que lorsque les livrables de la phase actuelle sont validés.

- On ne peut pas revenir à la phase précédente une fois celle-ci validée.

- Tous les besoins du projet sont exprimés et recueillis lors de la première phase.

- Le système est conceptualisé et validé avant la phase de développement.

- Les tests techniques et fonctionnels ne démarrent qu'après la phase de développements.

- L'intégration et la mise en production ont lieux après correction des anomalies.

Figure 6: Phases du cycle en « cascade »

On peut se rendre compte de la rigidité de cette approche. Cela peut être un atout dans un contexte dans lequel les besoins ne vont pas évoluer durant la vie du projet. Mais cela peut être un frein en matière de développement informatique. Par ailleurs, une anomalie dans la conception peut être décelée tardivement et entrainer un retard et dépassement du budget considérable.

Pour l'anecdote, Royce pointait déjà du doigt en 1970 les limites de cette approche et préconisait une approche itérative pour y pallier. Le fondateur du cycle de développement le plus classique était déjà une agilité ! Une variante du cycle en « cascade » est ainsi apparue afin d'identifier plus rapidement les anomalies en limitant le retour aux étapes précédentes. Il s'agit du cycle en « V ».

b. Le cycle en « V »

Le modèle du cycle en « V » a été élaboré pour éviter les problèmes de réactivité du cycle en cascade en cas d'anomalies. Recueil des besoins Spécifications Conception détaillée Codage Tests Intégration Le modèle est constitué d'une branche descendante qui comprend les étapes de conception du projet et d'une branche ascendante qui contient les étapes de tests. La jonction des

deux branches constitue la phase de codage du produit. Chaque étape d'une branche est en vis-à-vis avec l'étape de la branche opposée, les étapes de la branche montante doivent renvoyer de l'information à leur vis-à-vis lorsqu'une anomalie est détectée. Cela permet de déceler plus rapidement les problèmes que dans un cycle en cascade. Par ailleurs, cela offre la possibilité de préparer dans les étapes descendantes les besoins des étapes montantes. Ainsi les résultats attendus des tests unitaires seront définis dans la conception détaillée.

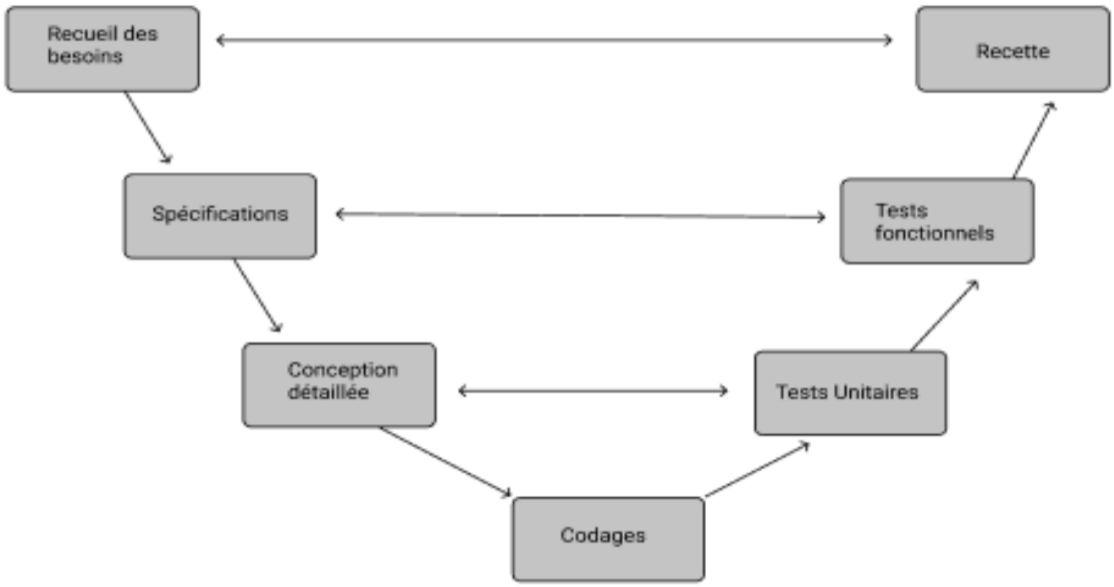

Figure 7: Phases du cycle en « V ».

c. Critiques des méthodes classiques

Nous allons lister les principaux.

Le risque de l'effet tunnel

C'est certainement un des principaux problèmes des méthodes classiques. Prenons l'exemple d'un projet qui dure six mois : l'entreprise fait un recueil des besoins auprès du client durant un mois, puis se lance dans la phase de développement durant quatre mois. Le client ne voit le résultat que le sixième mois… entre temps, les besoins de ce dernier ont pu changer et des fonctionnalités sont devenues obsolètes ou inadaptées. Recueil des besoins Spécifications Conception détaillée Codage Tests unitaires Tests fonctionnels Recette.

L'anticipation et la détection des risques

Comme nous venons de le voir dans les cycles en « cascade », les tests (unitaires et fonctionnels) sont réalisés après la phase de codage. Toutefois, l'impact d'une anomalie va augmenter avec l'avancement du projet. Les correctifs seront plus longs à implémenter et il sera nécessaire de s'assurer de la non régression du code.

La résistance au changement

Dans les méthodes classiques, les fonctionnalités à implémenter sont définies au lancement du projet, voire dans la réponse de l'entreprise dans le cas d'appel d'offres. Ces fonctionnalités sont détaillées dans le contrat initial entre le client et l'entreprise et toute modification de ces dernières est un sujet sensible car pouvant conduire à des dépassements de délais et de coûts. Dans le meilleur des cas, l'entreprise peut négocier l'amélioration d'une fonctionnalité contre l'abandon d'une autre… dans le pire des cas, le produit livré peut ne plus correspondre aux besoins du client. L'entreprise a donc tendance à refuser tous changements dans les demandes de modifications des fonctionnalités initiales. Nous allons maintenant présenter l'alternative offerte par les méthodes agiles et comprendre en quoi elles peuvent pallier les manques des méthodes classiques, que nous venons d'aborder.

2. Les méthodes agiles

A. Fondements théoriques

Avant de dresser un bref historique de l'apparition du mouvement agile, nous allons expliquer les principes fondamentaux de l'approche, puis nous présenterons les avantages et les bienfaits théoriques des méthodes agiles.

a. Définition d'une méthode agile

« Une méthode agile est une approche itérative et incrémentale, qui est menée dans un esprit collaboratif, avec juste ce qu'il faut de formalisme. Elle génère un produit de haute qualité tout en prenant en compte l'évolution des besoins des clients. ».

Deuff (2013) écrit que les méthodes agiles sont « des modèles itératifs et incrémentaux qui visent à répondre au mieux aux besoins exprimés par leurs commanditaires, en offrant une grande réactivité par rapport à leurs demandes » (Deuff & Cosquer, 2013). Nous retrouvons dans ces deux définitions les termes « itératif » et « incrémental ». Dans un développement itératif, le projet est découpé en plusieurs étapes d'une durée identique 29(en général quelques semaines), nommées « itérations ». A chaque fin d'itération, une version fonctionnelle du produit est attendue par le client pour validation, nous parlons d'une version intermédiaire du produit final et non d'un prototype. Les fonctionnalités sont ainsi progressivement intégrées au produit qui est conçu de façon « incrémentale »

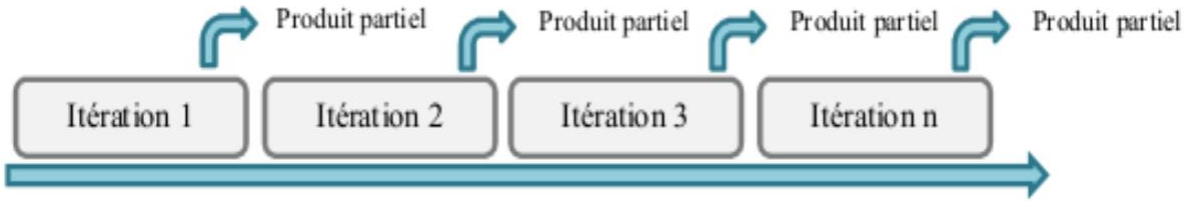

Figure 8: Représentation d'un développement itératif et incrémental

Les deux définitions mettent aussi l'accent sur la prise en compte de l'évolution des besoins du client. En effet, un des aspects fondamentaux des méthodes agiles est de partir du constat que les demandes des clients peuvent évoluer dans le cycle de vie du projet et qu'il est nécessaire de les prendre en compte pour livrer finalement le produit le plus adapté possible à leur besoin. Il est important de faire une distinction entre « méthode » et « pratique agile ». Une entreprise peut en effet décider de suivre certaines pratiques d'une méthode, sans adopter la méthode dans son ensemble. Elle peut aussi décider de mettre en œuvre des pratiques de plusieurs méthodes agiles, comme le souligne Boisvert (2011). Les méthodes agiles ont pour point commun de partager les valeurs et les principes consignés dans un document : le « manifeste agile ».

b. Qu'est-ce que l'agilité ? Valeurs et principes

Le mouvement agile voit son origine aux États-Unis, en février 2001, suite au constat du taux d'échec de plus en plus important des méthodes de gestion de projet classiques dans les années 1990 (Messager & Tabaka, 2010). Dix-sept experts du développement logiciel ayant pour la plupart mis au point de nouvelles méthodes se réunissent afin de débattre de nouvelles pratiques et de mettre en avant des valeurs communes (Fernandez et al., 2013). Du résultat de cette réflexion est né le terme « agile », ainsi que quatre valeurs inscrites dans le manifeste pour le développement agile de logiciels (Beck et al., 2001) : « Les individus et leurs interactions plus que les processus et les outils Des logiciels opérationnels plus qu'une documentation exhaustive La collaboration avec les clients plus que la négociation contractuelle L'adaptation au changement plus que le suivi d'un plan Nous reconnaissons la valeur des seconds éléments, mais privilégions les premiers. » De ces quatre valeurs découlent douze principes sous-jacents (Beck et al., 2001) (confère annexes page 102). Il est important de préciser que les deux méthodes agiles les plus utilisées ne sont pas apparues à la suite de cet événement. Les fondements de « Scrum » étaient présentés en 1995 lors de la conférence OOPSLA (Schwaber, 1995) et « Extreme Programming » a été lancé officiellement en 1999.

c. Avantages escomptés de l'agilité

Boisvert (2011) livre plusieurs avantages à l'adoption de l'agilité dans un projet :

- Fournir rapidement de la valeur au client : la valeur est apportée par les fonctionnalités validées par le client.

- Améliorer la qualité du produit en réduisant les défauts de conception (par les tests unitaires notamment).

- Réduire les délais de mise en production par des livraisons fréquentes du produit.

- Respecter le budget par la détection des risques le plus tôt possible et par la possibilité pour l'équipe de surveiller sa vélocité dans la réalisation des tâches.

- Améliorer la collaboration et la communication entre le client et l'entreprise.

d. Les deux principales méthodes

Une enquête nationale a été menée par Legardeur et Sutherland (2009) afin de connaitre les pratiques d'entreprises de tailles variées en matière de gestion de projet agile. Il en est notamment ressorti que les deux méthodes agiles les plus employées étaient Scrum et eXtreme Programming (XP). En troisième position arrive la méthode Lean. Cette dernière méthode, héritée du système de production de Toyota, n'est pas à proprement parler une méthode agile selon Messager (2010), même si certaines pratiques peuvent être employées dans le cadre d'une démarche agile. Nous centrerons notre présentation sur les deux principales méthodes agiles pour deux raisons :

- Ces deux méthodes sont les plus documentées et leur audience est la plus forte auprès de la communauté agile. Nous avons d'ailleurs pu nous en apercevoir lors de « l'agile tour Toulouse ».

- Nous le verrons dans la suite de ce mémoire, Scrum et XP sont deux méthodes très complémentaires, la première dans sa démarche de gestion de projet et la seconde pour ses pratiques d'ingénierie logicielle. Nous allons présenter chacune de ces deux méthodes selon leurs principes, le rôle des acteurs, les outils employés et leur mode opératoire. Nous n'avons pas pour objectif d'en faire une présentation exhaustive mais plutôt d'en fournir les notions essentielles. Nous nous appuierons sur les ouvrages de Fernandez (2013) et Messager (2010) pour les détailler.

2. La méthode Scrum

La méthode Scrum a été décrite pour la première fois dans un ouvrage par Schwaber et Beedle en (2001).

Valeurs

Scrum est une méthode qui se focalise sur la gestion de projet, elle ne couvre aucune pratique d'ingénierie logicielle. Elle repose sur trois valeurs :

La transparence : l'avancement du projet doit être visible des membres de l'équipe en charge et du propriétaire du produit.

Le contrôle : on doit pouvoir vérifier fréquemment les écarts de délais et de réalisation des tâches.

L'adaptation : on doit pouvoir faire des ajustements lorsque des écarts sont constatés afin de toujours répondre aux besoins du propriétaire du produit.

Rôles

La responsabilité de la conduite du projet est répartie sur trois rôles :

Le **product-owner** (propriétaire du produit ou PO) qui représente le client ou les utilisateurs finaux. Il va communiquer les besoins et la vision du produit à l'équipe de développement. Il sera impliqué à des moments clefs durant le développement.

L'équipe Scrum en charge du développement du produit. Elle est pluridisciplinaire et n'est pas hiérarchisée, les décisions sont prises de façon collégiale.

Le **Scrum-master** (animateur d'équipe) fait la liaison entre le product-owner et l'équipe Scrum, il peut faire partie de l'équipe et est en charge de la bonne mise en pratique de la méthode.

Outils

Il existe cinq principaux outils :

- ✓ Le product-backlog (carnet de produit) permet de lister les exigences données par le product-owner et classées par priorités. Le product-backlog peut être modifié en fonction de l'évolution des besoins du product-owner. Ces exigences sont appelées des user stories (récits d'utilisateur ou scénarios) et décrivent une fonctionnalité à développer.

Concrètement, les user stories sont des phrases simples qui répondent à trois interrogations : qui ? Quoi ? Pourquoi ? Voici un exemple : « En tant qu'utilisateur, je veux pouvoir rechercher un étudiant par son numéro afin de le retrouver rapidement. »

- ✓ Les user stories sont ensuite traduites et répertoriées dans le sprint-backlog (carnet de l'itération) sous forme de tâches à réaliser au cours d'une itération.
- ✓ Un graphique de progression, le Burndown chart, permet de visualiser l'avancement des tâches dans le temps. Il peut être complété par un graphe présentant la valeur acquise, c'est-à-dire les fonctionnalités validées par le client et qui lui sont immédiatement utiles.
- ✓ Le task board se présente sous la forme d'un tableau sur lequel on liste dans une première colonne les user stories à l'aide de post-it, puis les fonctionnalités du sprint-backlog dans quatre colonnes pour représenter leur avancement : à faire, en cours, à valider, fait.
- ✓ Enfin, la roadmap permet d'identifier les moments importants du projet, elle est mise à jour de façon régulière.

Pratiques

Un projet sous Scrum est découpé en itérations de quelques semaines (un mois en général) appelées Sprint. Au lancement du projet, les user stories du product-owner sont consignées dans le product-backlog. Avant chaque Sprint durant le sprint planning meeting, on sélectionne les user stories prioritaires qui seront développées et livrées en fin de Sprint.

Au cours du sprint, une réunion d'avancement a lieu de façon quotidienne avec les membres de l'équipe (le daily Scrum meeting ou mêlée quotidienne), afin de vérifier les écarts aux objectifs du Sprint. A la fin du Sprint, une version partielle mais opérationnelle du produit est présentée au product-owner durant le sprint review meeting ou revue de sprint. Le product-owner est censé pouvoir arrêter le projet s'il juge le produit répondant à ses besoins, dans le cas contraire, un nouveau Sprint est commandé.

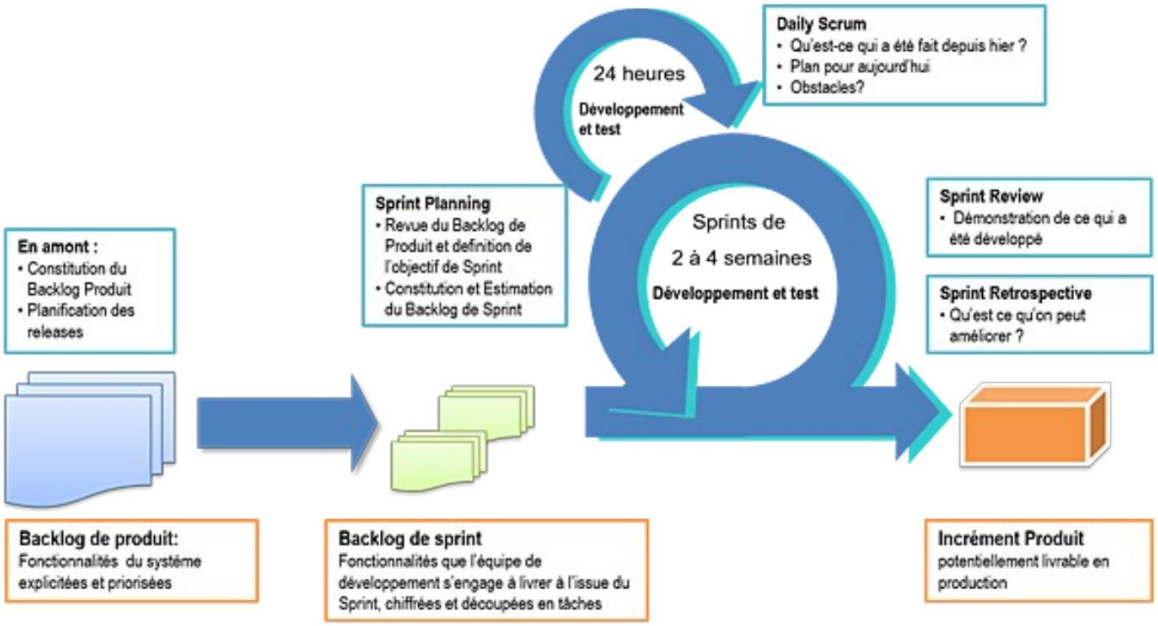

Figure 9: Le cycle de vie Scrum

2. La méthode eXtreme Programming (XP)

La méthode XP a été décrite dans l'ouvrage de Beck (2000).

Valeurs

XP est une méthode agile focalisée sur des pratiques d'ingénierie logicielle comme les tests unitaires et fonctionnels, voire le développement piloté par les tests unitaires (TDD), mais aussi la programmation en binôme ou encore l'intégration continue. Elle repose sur quatre valeurs :

La communication : il est indispensable que les intervenants d'un projet communiquent afin d'éviter des situations de blocage et la découverte tardive de problèmes.

La simplicité : la solution la plus simple est la meilleure. La simplicité doit se retrouver aussi bien dans les choix techniques que dans les besoins du client ou les processus à développer.

Le feedback : le retour d'information est indispensable d'un point de vue technique avec les tests unitaires ou fonctionnels pour valider le fonctionnement du produit et d'un point de vue organisationnel avec des livraisons fréquentes du produit pour validations du client.

Le courage : il en faut pour reprendre un code existant, faire de la programmation en binôme…

Rôles

Dans la littérature, à notre connaissance, les rôles de la méthode XP sont peu abordés, hormis dans l'ouvrage de Fernandez (2013) qui en distingue huit : le programmeur, le designer, le testeur, le tracker, le coach, le manager de projet, le client et le chef de produit. Par ailleurs, au sein de la communauté agile que nous avons pu côtoyer dans le cadre de ce mémoire, ce sont les pratiques de la méthode XP qui sont mises en avant plus que les rôles ou les outils.

En pratique, certains rôles sont confondus ; ainsi, le programmeur est souvent aussi le testeur et le client est généralement le chef de produit. Il faut retenir que la méthode XP fait la part belle aux activités de programmation plus qu'à la gestion de projet, qui s'avère moins formalisée qu'avec la méthode Scrum.

Outils

Il n'existe en fait que deux outils : les story-cards qui sont des cartes sur lesquelles on recense les user stories du client ; et le story-board visible en permanence par l'équipe, sur lequel on affiche les story-cards. On précise dans chaque story-card les ressources nécessaires pour l'accomplissement de ces user stories ainsi que les tests associés. Ces story-cards peuvent être complétées par des post-it pour visualiser l'avancement du projet, à la manière du task board de la méthode Scrum.

Pratiques

Contrairement à la méthode Scrum, la méthode XP ne se focalise pas seulement sur la gestion de projet mais propose des pratiques et des outils très poussés pour améliorer la collaboration entre les développeurs et optimiser la programmation. Il est à noter qu'XP offre une description des processus de gestion de projet moins détaillée que celle de Scrum. XP fonctionne comme Scrum, selon le principe de cycles rapides de développement, les itérations. Un projet XP est composé d'itérations qui s'articulent selon les étapes suivantes :

- ✓ Au lancement d'une itération, le client et l'équipe de développement consignent dans des story-cards les user stories à développer, cela se déroule lors d'un planning poker.
- ✓ Chacune des user stories est transformée en tâche qui seront réalisées durant l'itération selon les pratiques collaboratives et de programmation que nous allons aborder en suivant (TDD, intégration continue…).
- ✓ A la fin de l'itération, si les user stories réalisées sont validées et que le client fournit de nouveaux scénarios à développer, une nouvelle itération démarre. Il est à noter que le client possède une forte implication dans le projet et est idéalement présent sur le site pour les réunions de planning poker notamment.

Les principales pratiques collaboratives de la méthode XP sont :

- ✓ Le pair programming (programmation en binôme) dont le principe est de faire travailler durant un laps de temps assez court (une heure par exemple) deux développeurs sur une machine afin de produire un code de plus grande qualité. Cette pratique n'est possible qu'en respectant un coding standard (règles de codage) commun pour l'équipe.
  - ✓ L'intégration continue qui consiste à intégrer au produit les modifications faites par les développeurs de façon continue (au moins quotidiennement) afin de détecter les problèmes au plus tôt grâce à des tests d'intégration.

Les pratiques relatives à la programmation sont les suivantes. Le simple design (conception simple) proposé par Beck (2000) consiste en quatre règles qu'un programmeur doit respecter en écrivant son code :

- ✓ Passer tous les tests.
- ✓ Ne pas dupliquer le code.
- ✓ Révéler ses intentions par le code.
- ✓ Réduire le nombre de classes et de méthodes.

Le code source doit être soumis à des tests unitaires et la méthode propose la pratique du Test Driven Development (TDD). Le TDD consiste à écrire le test unitaire avant le code. Il se déroule suivant un cycle en cinq étapes :

- ❖ Ecrire un premier test.
- ❖ Lancer le test qui doit échouer pour vérifier qu'il est valide (en effet le code n'existe pas encore). - Ecrire le code permettant de faire passer le test.
- ❖ Vérifier que le test passe.
- ❖ Refactoriser le code, c'est-à-dire l'améliorer en suivant par exemple les règles du simple design.

Les tests unitaires pourront être suivis par des tests fonctionnels afin de valider les fonctionnalités développées.

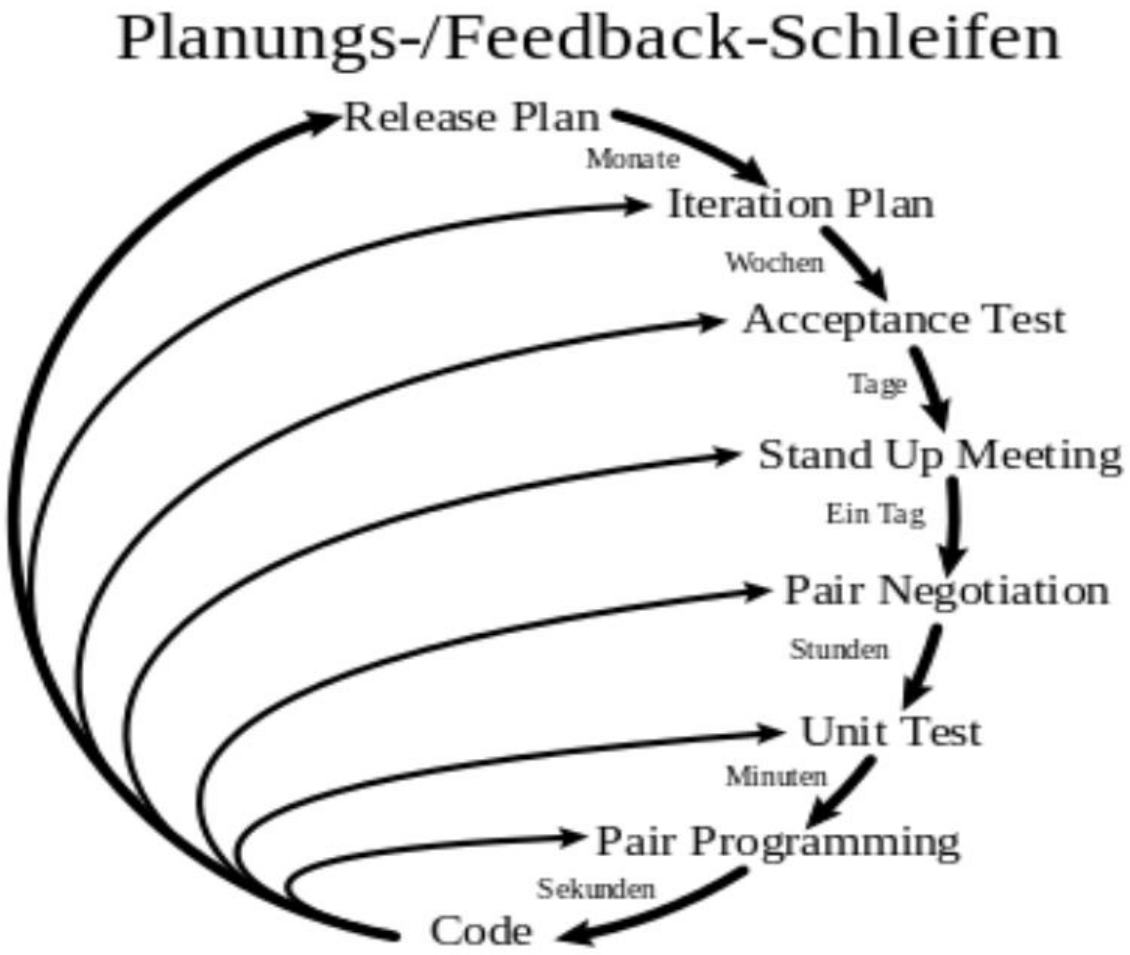

Figure 10: Le cycle de vie eXtreme programming (XP) (Valentine, 2011)

3. Comparaison des méthodes Scrum et eXtreme Programming (XP)

En nous appuyant sur les ouvrages de Fernandez (2013) et de Messager (2010), nous pouvons mettre en avant les forces et les faiblesses des deux méthodes.

Tableau 2: Comparaison des méthodes Scrum et eXtreme Programming (XP)

Méthode	Points forts	Points faibles
Scrum	• Se focalise sur la création de valeur ajoutée pour le product-owner. • Favorise la communication et la collaboration au sein de l'équipe et avec le product-owner. • Propose un cadre de travail qui peut être utilisé sur des projets autres qu'informatiques.	• N'aborde pas de pratiques d'ingénierie logicielle. • Changements limités avec des itérations de trente jours et des tâches à réaliser figées durant celles-ci.

EXtreme Programming (XP)	• Propose un programme de certification. Ne nécessite pas d'outillage complexe pour sa mise en place. Solides pratiques d'ingénierie logicielle. Les tests présents à tous les niveaux du développement, favorisent l'émergence de qualité du code. Des itérations courtes et une implication importante du client, donc un feedback fréquent.	Demande de l'expérience et du temps pour développer certaines pratiques techniques : TDD, intégration continue. Le pair programming n'est pas toujours apprécié des développeurs. Nécessite beaucoup de rigueur, notamment dans la conduite des tests. Faible documentation, méthode limitée dans le cas d'un projet évolutif ou multi équipes.

Nous pouvons voir que certains points faibles d'une méthode sont les points forts de l'autre. De là à dire que ces méthodes peuvent être complémentaires, il n'y a qu'un pas…

4. Vers une hybridation des deux méthodes

Nous l'avons vu, Scrum et XP proposent chacune leur façon d'aider l'équipe. Alors que Scrum propose des activités permettant d'améliorer la gestion de projet, XP propose des outils et des pratiques d'ingénierie logicielle. Nous retenons de la comparaison entre ces deux méthodes leur grande complémentarité et les avantages que nous pouvons escompter en combinant leurs pratiques. Selon Boisvert (2011), la combinaison des méthodes agiles est très répandue, leurs pratiques sont compatibles comme dans le cas de Scrum et XP et permettent en se complétant d'obtenir une méthode agile hybride adaptée à une organisation ou un projet.

3. Comparaison entre approche classique et approche agile

Nous venons de voir que les méthodes agiles offrent une approche bien différente de celle proposée dans les méthodes classiques. Le tableau ci-dessous s'appuie sur l'ouvrage de Messager (2010) et nous permet de faire une synthèse comparative des deux approches.

Tableau 3: Comparaison entre approche classique et approche agile

Thème	**Approche classique**	**Approche agile**
Cycle de vie	Séquentiel, en cascade ou en V sans rétroaction possible.	Itératif et incrémental.
Planification	Prédictive, les besoins sont définis dès le lancement du projet et les développements planifiés sans modification possible.	Adaptative, des ajustements peuvent être effectués à chaque fin d'itérations.
Changement	Résistance au changement.	Intégré dans l'approche.
Equipe	Une équipe avec des ressources Spécialisées. Dirigée par le chef de projet.	Une équipe responsabilisée où L'initiative et la communication sont encouragés. Soutenue par le chef de projet.
Qualité	Contrôle qualité tardif, à la fin du cycle de développement. Le client découvre le produit fini.	Contrôle qualité régulier. Le client visualise les résultats tôt et fréquemment.
Suivi de l'avancement	Mesure de la conformité aux plans initiaux. Analyse des écarts à posteriori.	Deux indicateurs d'avancement : le travail restant à faire et la valeur acquise par le client (fonctionnalités utiles au client, implémentées et fonctionnelles).
Documentation	Produite en quantité importante comme support de communication, devalidation et de contractualisation.	Réduite au strict nécessaire au profit d'incréments opérationnels du produit et d'échanges fréquents avec le client.
Gestion des risques	Processus distinct de gestion des risques.	Intégré dans le processus global, avec responsabilisation de chacun (équipe et client) dans l'identification et la résolution des risques
Mesure du succès	Respect des engagements initiaux en termes de coûts, de budget et de qualité.	Satisfaction du client par la livraison de valeur ajoutée.

Pour donner suite à ce que nous venons de voir, l'agilité semble être une alternative séduisante, voire incontournable, aux approches classiques.

4. La méthode utilisée dans ce projet

Blacklistage. Les outils structurés de Scrum ont permis de gérer efficacement les délais, les priorités et la communication avec les parties prenantes, tandis que les pratiques techniques de XP ont assuré une qualité élevée du code et une collaboration fluide au sein de l'équipe.

Cette approche hybride a également favorisé une adaptation rapide aux changements et une réactivité accrue face aux retours des utilisateurs. Par exemple, lorsque des ajustements ont été nécessaires pour répondre à des critères spécifiques (comme la durée de blocage ou le nombre d'appels autorisés), l'équipe a pu les intégrer rapidement grâce à la flexibilité de la méthode agile.

Nous avons aussi pour les rôles, le testeur pour la validation de chaque sprint-backlog, le coach dans chaque équipe, le manager du projet, le client et le chef de projet. Comme dans la méthode XP, le client est le chef projet, le tester et le manager sont aussi des coachs.

Pour la partie web, nous avons commencé par le sprint de la création de la page d'authentification en respectant la maquette du design, ensuite le sprint de visualisation des différents types de réunions et les sprints de choisir sa mode de présence et de gestion des notifications enfin les sprints de visualisation des contenus des conseils d'administration et la lecture des documents des différentes procés-verbaux. Et à la fin de chaque sprint, des tests sont réalisés et des feedbacks permettent de corriger ou d'améliorer le code.

IV. Conception détaillées

IV.1. Les diagrammes des cas d'utilisation.

V.1.1. Définition

Les rôles des diagrammes de cas d'utilisation sont de recueillir, d'analyser et d'organiser les besoins, ainsi que de recenser les grandes fonctionnalités d'un système. Il s'agit donc de la première étape UML pour la conception d'un système. Un diagramme de cas d'utilisation capture le comportement d'un système, d'un sous-système, d'une classe ou d'un composant tel qu'un utilisateur extérieur le voit. Il scinde la fonctionnalité du système en unités cohérentes, les cas d'utilisation, ayant un sens pour les acteurs. Ainsi ces cas d'utilisation permettent d'exprimer le besoin des utilisateurs d'un système, ils sont donc une vision orientée utilisateur de ce besoin au contraire d'une vision informatique.

Il ne faut jamais négliger cette première étape pour produire un site web conforme aux attentes des utilisateurs ciblés. Pour élaborer les cas d'utilisation, il faut se fonder sur des entretiens avec les utilisateurs.

IV.1.2. Composition du diagramme de cas d'utilisation

Le diagramme de cas se compose de trois éléments principaux : Le diagramme de cas se compose de trois éléments principaux :

Un Acteur : c'est l'idéalisation d'un rôle joué par une personne externe, un processus ou une chose qui interagit avec un système. Il se représente par un petit bonhomme avec son nom inscrit dessous.

Figure 11: acteur

Un cas d'utilisation : c'est une unité cohérente représentant une fonctionnalité visible de l'extérieur. Il réalise un service de bout en bout, avec un déclenchement, un déroulement et une fin, pour l'acteur qui l'initie. Un cas d'utilisation modélise donc un service rendu par le système, sans imposer le mode de réalisation de ce service. Il représente par une ellipse contenant le nom du cas (un verbe à l'infinitif), et optionnellement, au-dessus du nom, un stéréotype.

Les relations : Trois types de relations sont pris en charge par la norme UML et sont graphiquement représentées par des types particuliers de ces relations. Les relations indiquent que le cas d'utilisation source présente les mêmes conditions d'exécution que le cas issu. Une relation simple entre un acteur et une utilisation est un trait simple.

IV.1.2. Les acteurs de notre projet

L'administrateur : C'est l'acteur principal, celui qui assure le dynamisme du site et veille sur les mises à jour.

IV.1.4. Diagrammes de cas d'utilisation de notre application

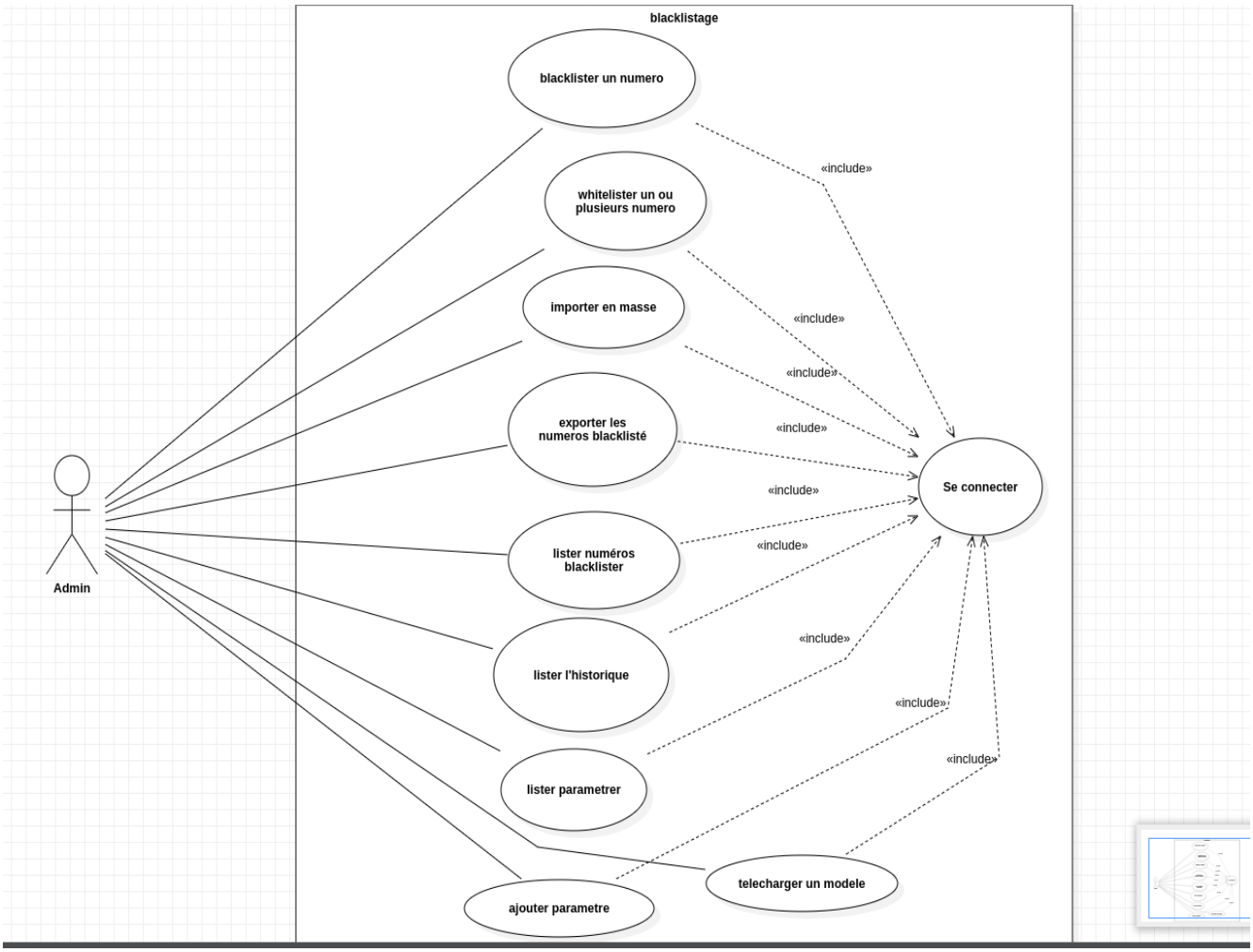

Figure 12: Diagrammes de cas d'utilisation de notre application

Tableau - Use-case "se connecter"

Tableau 4: Use-case "se connecter"

Nom	**se connecter**
Acteur	Admin
Déclencheur	L'acteur clique sur le bouton « Se connecter »
Préconditions	L'acteur s'est rendu sur la page de connexion
Flux Principale	1. L'utilisateur clique sur le bouton « Se connecter » 2. Les informations de connexion sont envoyées à l'API 3. L'API définit si les informations reçues sont correctes 4. L'API créé un token et le retourne 5. Le token est enregistré localement sur la machine 6. La page « Blackliste » s'affiche
Flux Secondaire	Les informations reçues sont correctes Retour au flux N° 4 L'utilisateur n'a pas validé son compte L'API retourne un message d'erreur Le message d'erreur est affiché Retour au flux N° 1 Les informations de connexion ne sont pas correctes L'API retourne un message d'erreur Le message d'erreur est affiché Retour au flux N° 1

Tableau - Use-case "blacklister un numéro"

Tableau 5: Use-case "blacklister un numéro"

Élément	**Description**
Nom	**Blacklister un numéro**
Acteur	Admin
Déclencheur	L'acteur clique sur le bouton "Ajouter" après avoir saisi un numéro et sélection un service
Préconditions	-L'acteur est connecté avec ses droits d'administrateur - L'acteur est sur la page "Blackliste"
Déclencheur	
Flux Principal	1. L'administrateur saisit le numéro dans le champ prévu 2. L'administrateur sélectionne un service dans le menu déroulant 3. L'administrateur clique sur le bouton "Ajouter" 4. Le système enregistre le numéro dans la base de données

	5. Le numéro apparaît dans la liste des numéros blacklistés
Flux Secondaire	Si le format du numéro n'est pas bon: - Le système affiche un message d'erreur - Retour au flux N° 1 Si les champs obligatoires ne sont pas remplis: - Le système affiche un message d'erreur - Retour au flux N° 1

Tableau - Use-case " Importation en masse”

Tableau 6: Use-case " Importation en masse”

Élément	Description
Nom	**Importation en masse**
Acteur	Admin
Déclencheur	L'acteur clique sur le bouton "Importer maintenant"
Préconditions	-L'acteur est connecté avec ses droits d'administrateur - L'acteur est sur la page "Blackliste"
Flux Principal	1. L'administrateur clique sur la zone pour sélectionner un fichier 2. L'administrateur sélectionne un fichier à importer 3. L'administrateur clique sur "Importer maintenant" 4. Le système traite le fichier et importe les numéros 5. une fenêtre de confirmation s’ouvre 6. Les numéros importés apparaissent dans la liste des numéros blacklistés
Flux Secondaire	Si le format du fichier est incorrect: - Le système affiche un message d'erreur - Retour au flux N° 1 - Fin du cas d'utilisation

Tableau - Use-case "Télécharger le modèle**”**

Tableau 7: Use-case "Télécharger le modèle”

Élément	Description
Nom	Télécharger le modèle
Acteur	Admin
Déclencheur	L'acteur clique sur le bouton "Importer maintenant"
Préconditions	-L'acteur est connecté avec ses droits d'administrateur - L'acteur est sur la page "Blackliste"
Flux Principal	1. L'administrateur clique sur "Télécharger le modèle" 2. Le système génère un fichier modèle pour l'importation en masse 3. Le système propose le téléchargement du fichier 4. L'administrateur télécharge le fichier sur son poste

Flux Secondaire	Si une erreur survient durant la génération du modèle: - Le système affiche un message d'erreur - Fin du cas d'utilisation

Tableau - Use-case "Lister numéros blacklistés**”**

Tableau 8: Use-case "Lister numéros blacklistés”

Élément	Description
Nom	Lister numéros blacklistés
Acteur	Admin
Déclencheur	L'acteur clique sur le bouton "Importer maintenant"
Préconditions	-L'acteur est connecté avec ses droits d'administrateur - L'acteur est sur la page "Blackliste"
Flux Principal	1. L'administrateur accède à la page "Blackliste" 2. Le système affiche la liste complète des numéros blacklistés 3. L'administrateur peut consulter les informations pour chaque numéro (Numéro, Service, Date, Origine)
Flux Secondaire	Si aucun numéro n'est présent dans la liste: - Le système affiche un message indiquant que la liste est vide - Fin du cas d'utilisation

Tableau - Use-case "Ajouter Paramètre"

Tableau 9: Use-case "Ajouter Paramètre"

Élément	Description
Nom	**Ajouter Paramètre**
Acteur	**Admin**
Déclencheur	**L'acteur clique sur le bouton " Ajouter"**
Préconditions	**- L'acteur est connecté avec ses droits d'administrateur** **- L'acteur est sur la page "Gestion des Paramètres"**
Flux Principal	**1. L'administrateur clique sur le bouton "Ajouter"**

	2. Le système affiche un formulaire de création de paramètre **3. L'administrateur saisit le service** **4. L'administrateur définit le seuil** **5. L'administrateur spécifie la durée** **6. L'administrateur indique la période** **7. L'administrateur valide le formulaire** **8. Le système enregistre le nouveau paramètre** **9. Le paramètre apparaît dans la liste des paramètres**
Flux Secondaire	**Si le service existe déjà avec les mêmes paramètres :** **- Le système affiche un message d'erreur** **- Retour au flux N° 3** **Si les champs obligatoires ne sont pas remplis :** **- Le système affiche un message d'erreur** **- Retour aux champs du formulaire**

Tableau - Use-case " Lister paramètres”

Tableau 10: Use-case " Lister paramètres”

Élément	Description
Nom	**Lister paramètres**
Acteur	Admin
Déclencheur	L'acteur clique sur l'onglet "Paramètres"
Préconditions	- L'acteur est connecté avec ses droits d'administrateur - L'acteur est sur la page "Gestion des Paramètres"
Flux Principal	1. L'administrateur clique sur l'onglet "Paramètres" dans le menu principal 2. Le système affiche la liste des paramètres configurés 3. L'administrateur peut consulter les informations pour chaque paramètre (Service, Seuil, Durée, Période)

Flux Secondaire	Si aucun paramètre n'est configuré: - Le système affiche un message indiquant qu'aucun paramètre n'est défini - Fin du cas d'utilisation

Tableau – Use-case « Whitelister la sélection »

Tableau 11: Use-case « Whitelister la sélection »

Élément	Description
Nom	**Télécharger le modèle**
Acteur	**Admin**
Déclencheur	**L'acteur clique sur le bouton " Whitelister la sélection "**
Préconditions	**-L'acteur est connecté avec ses droits d'administrateur** **- L'acteur est sur la page "Blackliste"**
Flux Principal	**1. L'administrateur sélectionne un ou plusieurs numéros dans la liste** **2. L'administrateur clique sur "Whitelister la sélection"** **3. Le système retire les numéros sélectionnés de la blackliste** **4. Les numéros ne sont plus affichés dans la liste des numéros blacklistés**
Flux Secondaire	**Si aucun numéro n'est sélectionné :** **- Le bouton whitelister est desactiver** **- Fin du cas d'utilisation**

Tableau - Use-case "Exporter"

Tableau 12: Use-case "Exporter"

Élément	Description
Nom	**Télécharger le modèle**
Acteur	**Admin**
Déclencheur	**L'acteur clique sur le bouton " Exporter "**

Préconditions	**-L'acteur est connecté avec ses droits d'administrateur** **- L'acteur est sur la page "Blackliste"**
Flux Principal	**1. L'administrateur clique sur "Exporter"** **2. Le système génère un fichier contenant tous les numéros blacklistés** **3. Le système propose le téléchargement du fichier** **4. L'administrateur télécharge le fichier sur son poste**
Flux Secondaire	**Si aucun numéro n'est présent dans la liste:** **- Le système génère un fichier vide** **- Retour au flux N° 3** **Si une erreur survient durant la génération :** **- Le système affiche un message d'erreur** **- Fin du cas d'utilisation**

Tableau - Use-case " Lister l'historique "

Tableau 13: Use-case " Lister l'historique"

Élément	Description
Nom	Télécharger le modèle
Acteur	Admin
Déclencheur	L'acteur clique sur le bouton " **Exporter** "
Préconditions	-L'acteur est connecté avec ses droits d'administrateur - L'acteur est sur la page "Blackliste"
Flux Principal	1. L'administrateur clique sur "Exporter" 2. Le système génère un fichier contenant tous les numéros blacklistés 3. Le système propose le téléchargement du fichier 4. L'administrateur télécharge le fichier sur son poste
Flux Secondaire	Si aucun numéro n'est présent dans la liste :

	- Le système génère un fichier vide - Retour au flux N° 3 Si une erreur survient durant la génération : - Le système affiche un message d'erreur - Fin du cas d'utilisation

IV.2. Le diagramme de classe

IV.2.1. Définition

Un diagramme de classes UML décrit les structures d'objets et d'informations utilisées sur notre site web, à la fois en interne et en communication avec ses utilisateurs. Il décrit les informations sans faire référence à une implémentation particulière. Ses classes et relations peuvent être implémentées de nombreuses manières, comme les tables de bases de données, les nœuds XML ou encore les compositions d'objets logiciels.

IV.2.2. La composition d'un diagramme de classes

En général un diagramme de classe peut contenir les éléments suivants :

Les classes : une classe représente la description formelle d'un ensemble d'objets ayant une sémantique et des caractéristiques communes. Elle est représentée en utilisant un rectangle divisé en trois sections. La section supérieure est le nom de la classe, la section centrale définit les propriétés de la classe alors que la section du bas énumère les méthodes de la classe.

Les associations : une association est une relation entre deux classes (association binaire) ou plus (association n-aire), qui décrit les connexions structurelles entre leurs instances. Une association indique donc que des liens peuvent exister entre des instances des classes associées.

Les attributs : les attributs représentent les données encapsulées dans les objets des classes. Chacune de ces informations est définie par un nom, un type de données, une visibilité et peut-être initialisé. Le nom de l'attribut doit être unique dans la classe.

IV.2.3. Diagramme de classe de l'application

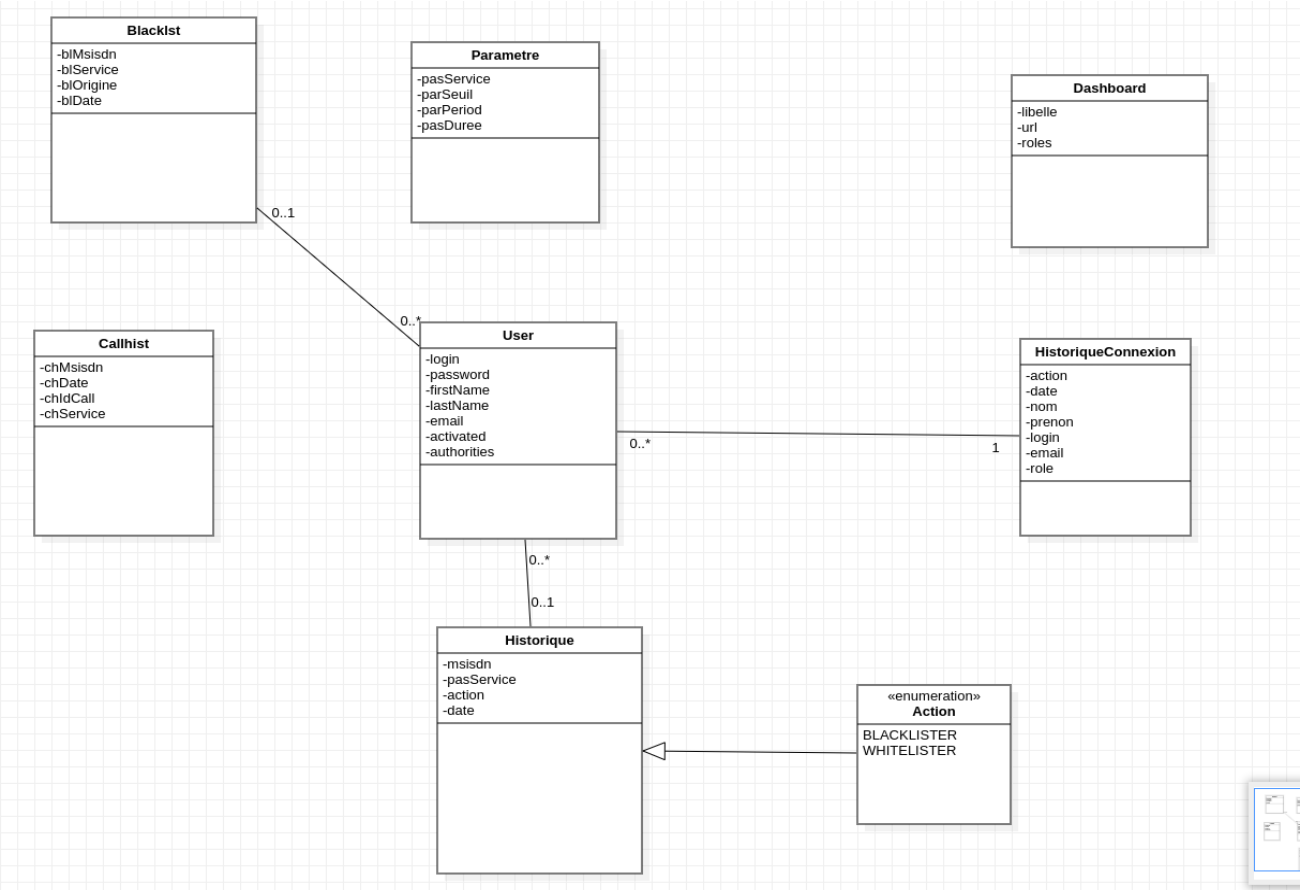

Figure 13: Diagramme de classe de l'application

IV.3. Les diagrammes des Séquence

IV.3.1 Définition

Un diagramme de séquences est un diagramme d'interaction qui expose en détail la façon dont les opérations sont effectuées : quels messages sont envoyés et quand ils le sont. Les diagrammes de séquences sont organisés en fonction du temps qui s'écoule au fur et à mesure que nous parcourons la page. Les objets impliqués dans l'opération sont répertoriés de gauche à droite en fonction du moment où ils prennent part dans la séquence.

IV.3.2. Composition d’un diagramme de séquences

Ce type des diagrammes est composé par les éléments suivants :

Les lignes de vie : Une ligne verticale qui représente la séquencedes événements, produite par un participant, pendant une interaction, alors que le temps progresse en bas de ligne. Ce participant peut être une instance d'une classe, un composant ou un acteur.

Les messages : deux types de messages dans le diagramme de séquences, le premier est dit message synchrone utilisé pour représenter des appels de fonction ordinaires dans un programme, le deuxième est appelé message asynchrone, étant utilisé pour représenter la communication entre des threads distincts ou la création d'un nouveau thread.

Les occurrences d'exécution : représente la période d’exécution d’une opération.

Les commentaires : Un commentaire peut être joint à tout point sur une ligne de vie.

Les itérations : représente un message de réponse suite à une question de vérification.

IV.3.3. Diagramme de séquences d’authentification

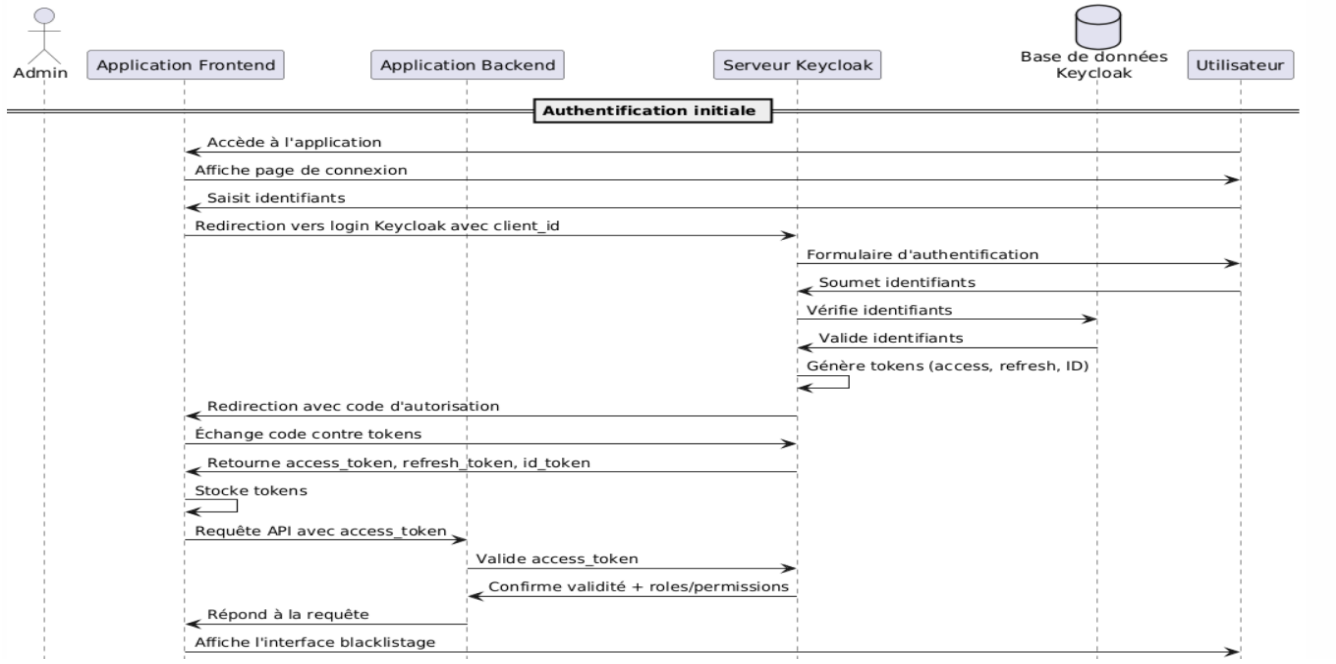

Figure 14: Diagramme de séquence d'authentification

IV.3.4 Diagramme de séquence blacklister un numéro

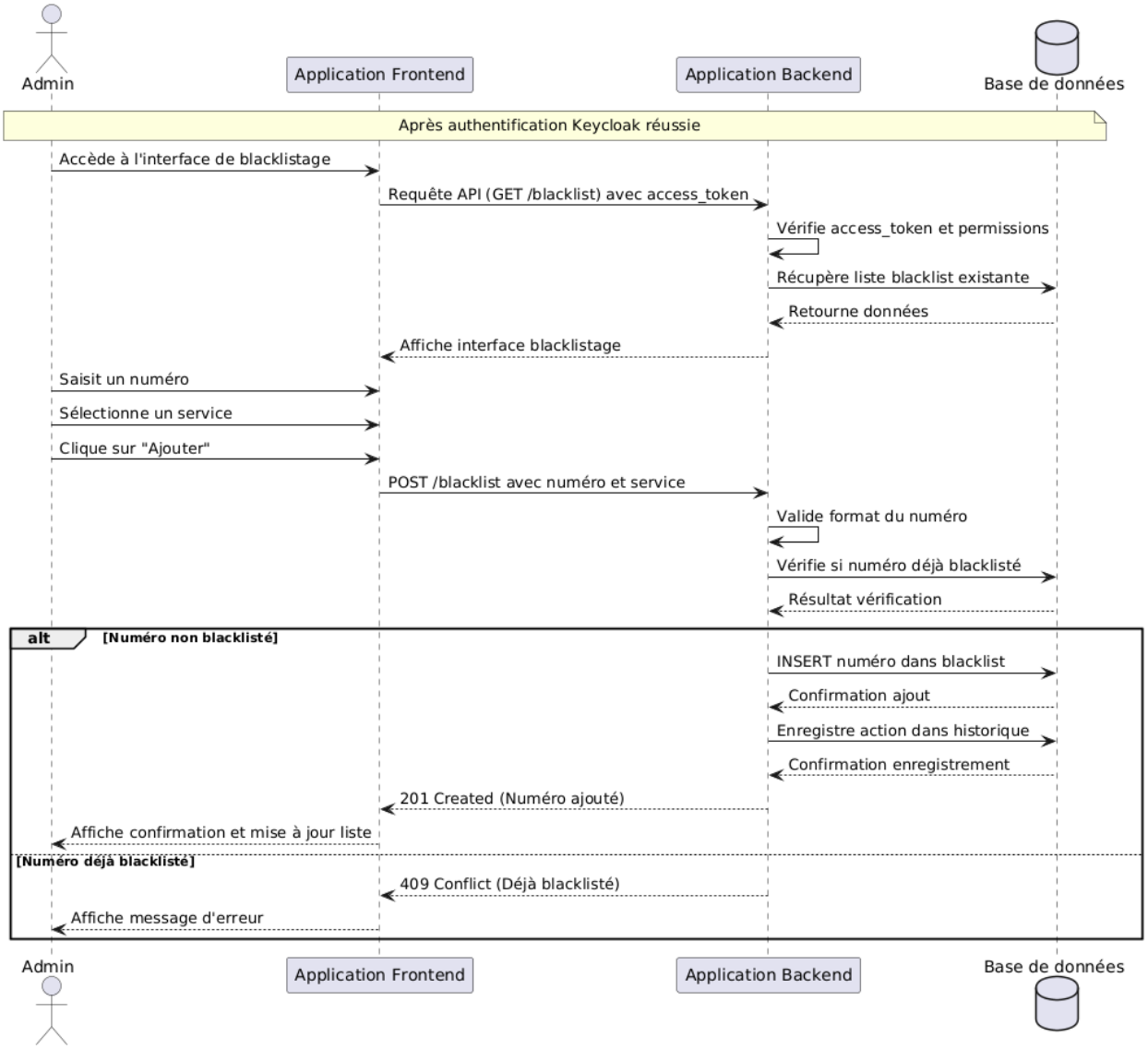

Figure 15: Diagramme de séquence blacklister un numéro

IV.3.5 Diagramme de séquence blacklister un numéro

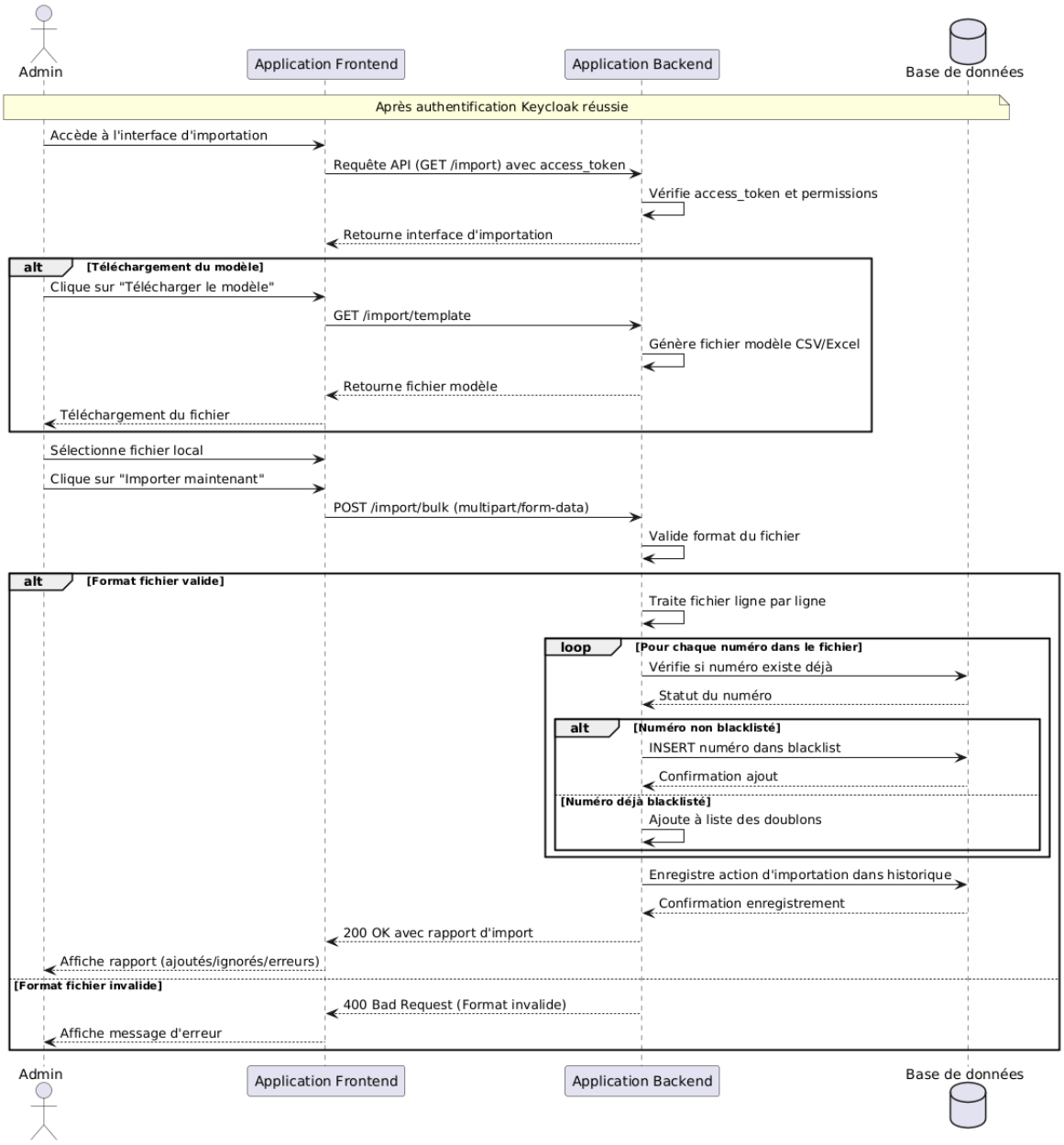

Figure 16: Diagramme de séquence blacklister un numéro

IV.4 Prototypage de la solution

Le prototypage est de nos jours une étape importante qui s'impose entre la phase de conception d'une solution et de sa réalisation. En effet il permet d'avoir une conception détaillée d'une application ou d'un site, il permet de renseigner à quoi ressemblera l'expérience finale.

Afin de répondre à nos besoins spécifiés précédemment, notamment en termes de convivialité nous avons opter d'utiliser Figma pour faire la représentation visuelle (design) de notre application ainsi que le prototypage c'est-à dire montrer à la fois le contenu visuel et les interactions entre les éléments (boutons, pop-up, menus...).

IV.4.1 Présentation de Figma

Figma est une plateforme collaborative pour éditer des graphiques vectoriels et faire du prototypage. Elle permet de concevoir des design systems pour faciliter la création de sites web et d'applications mobiles. C'est une solution à destination des UI et UX designers et des développeurs. L'interface propose de nombreuses fonctionnalités :

- de design : avec des outils de conception pour le web, des fonctions de mise en page automatique, des plugins pour réduire les tâches répétitives ;

Figure 17: Figma

IV.4.2 Présentation de quelques captures de mes prototypes

Par défaut la page d'accueil de notre application se présentera comme l'écran ci-dessous paramétrage.

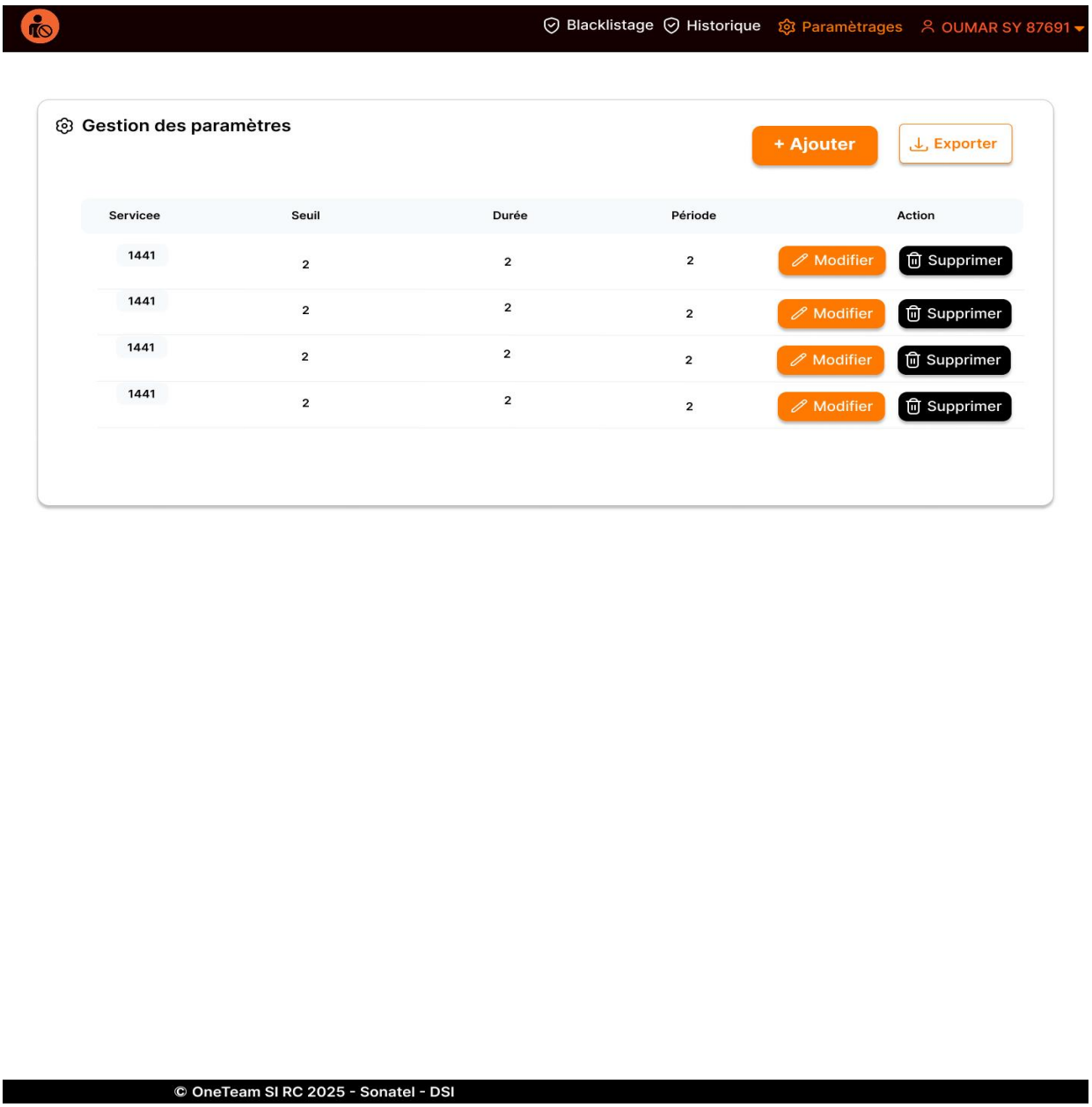

Figure 18: Prototype paramètre

Voici le prototype de blacklistage

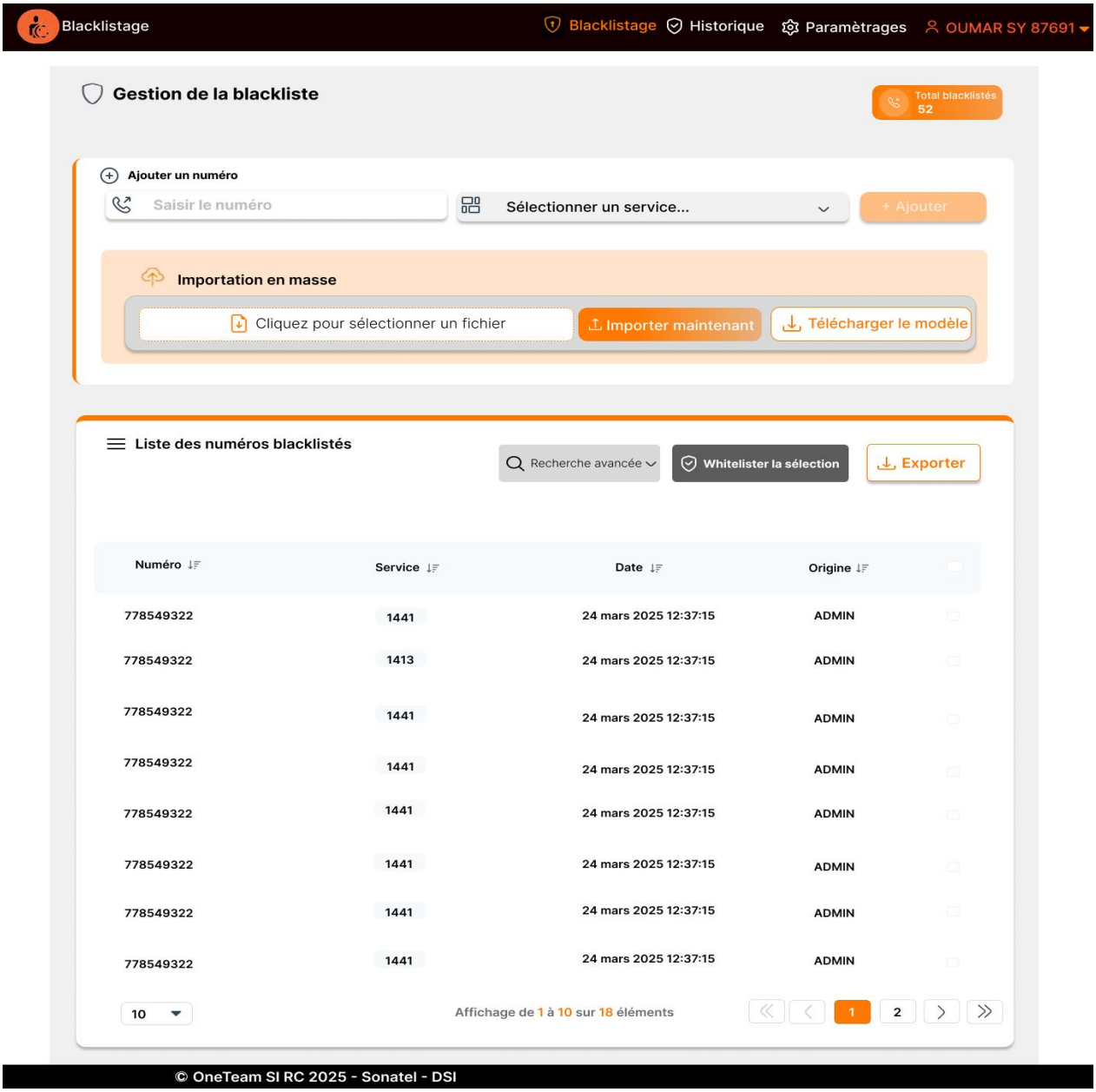

Figure 19: Prototype blacklistage

Voici le prototype de l’historique

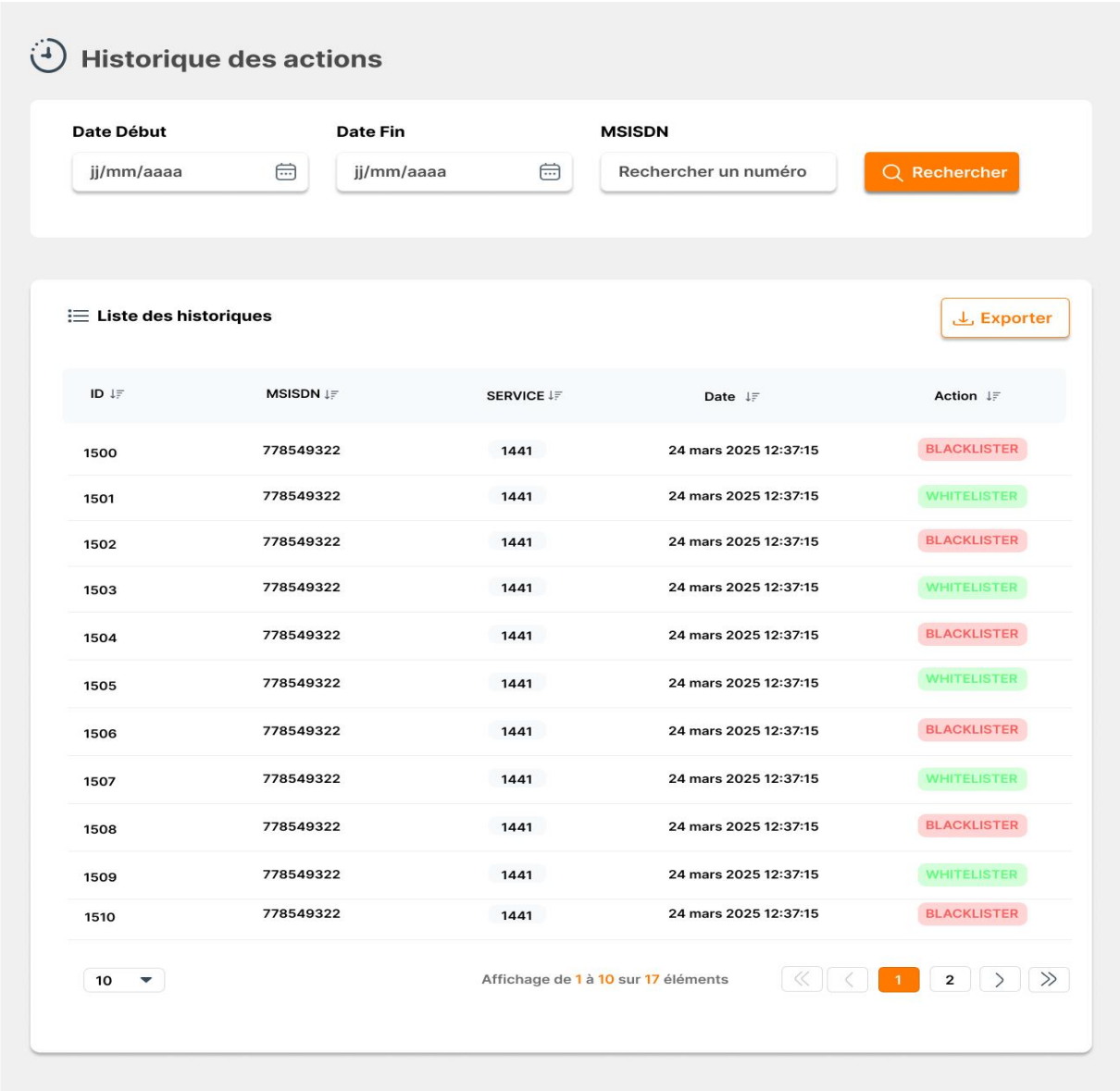

Figure 20: Prototype de l'historique

V. Le développement de l’application

V.1 Méthodologie de gestion de projet

Pour mener à bien le développement de notre solution de gestion des appels répétitifs, nous avons opté pour une méthodologie Agile, plus précisément la méthode Scrum. Cette approche présente plusieurs avantages pour notre projet :

- **Flexibilité et adaptation** : La nature du projet, qui vise à automatiser la gestion des appels répétitifs, nécessite une capacité d'adaptation rapide aux retours des utilisateurs finaux et aux évolutions des besoins métier.
- **Livraisons itératives** : La méthode Scrum nous permet de livrer des fonctionnalités opérationnelles à intervalles réguliers (toutes les 2 semaines), permettant ainsi aux utilisateurs de tester rapidement les modules développés et de fournir un retour d'expérience précieux.
- **Communication continue** : Les rituels Scrum (daily meetings, sprint planning, sprint review, rétrospective) ont favorisé une communication constante entre les membres de l'équipe de développement et les parties prenantes de Sonatel.

Notre cycle de développement a été structuré en sprints de deux semaines, chaque sprint visant à produire un incrément fonctionnel de notre solution de blacklistage.

V.2 Utilisation de l'application GDD pour la gestion de projet

V.2.1 Présentation de l'outil GDD

GDD (Gestion Des Développements) est l'outil interne adopté pour la gestion de ce projet. Cette plateforme propriétaire, développée spécifiquement pour les besoins de l'entreprise, permet une visualisation claire de l'avancement du projet à travers un système de tableaux et de cartes similaire à celui de Kanban.

L'interface de GDD, visible sur la première capture d'écran, offre une vue structurée des épics (grandes fonctionnalités) et des tâches associées. Chaque produit, comme notre solution de blacklistage des numéros à répétition d'appels (Prod_1508), dispose de son propre backlog et tableau de bord pour suivre l'avancement.

V.2.2 Comparaison avec Trello

GDD présente des similarités avec l'outil Trello, mais avec des fonctionnalités adaptées spécifiquement aux besoins de développement de notre organisation :

Tableau 14: Comparaison GDD vsTrello

Caractéristique	GDD	Trello
Organisation hiérarchique	Structure produits/épics/tâches	Structure tableaux/listes/cartes
Suivi des statuts	États personnalisés (Non démarré, En cours, Terminés)	Colonnes personnalisables
Intégration métier	Adaptée aux processus internes	Générique et flexible
Reporting	Tableaux de bord intégrés avec métriques spécifiques	Extensions nécessaires pour des métriques avancées
Gestion des versions	Intégrée	Limitée sans extensions

Cette comparaison met en évidence l'adéquation de GDD avec nos processus internes, facilitant ainsi le suivi du projet selon nos standards organisationnels.

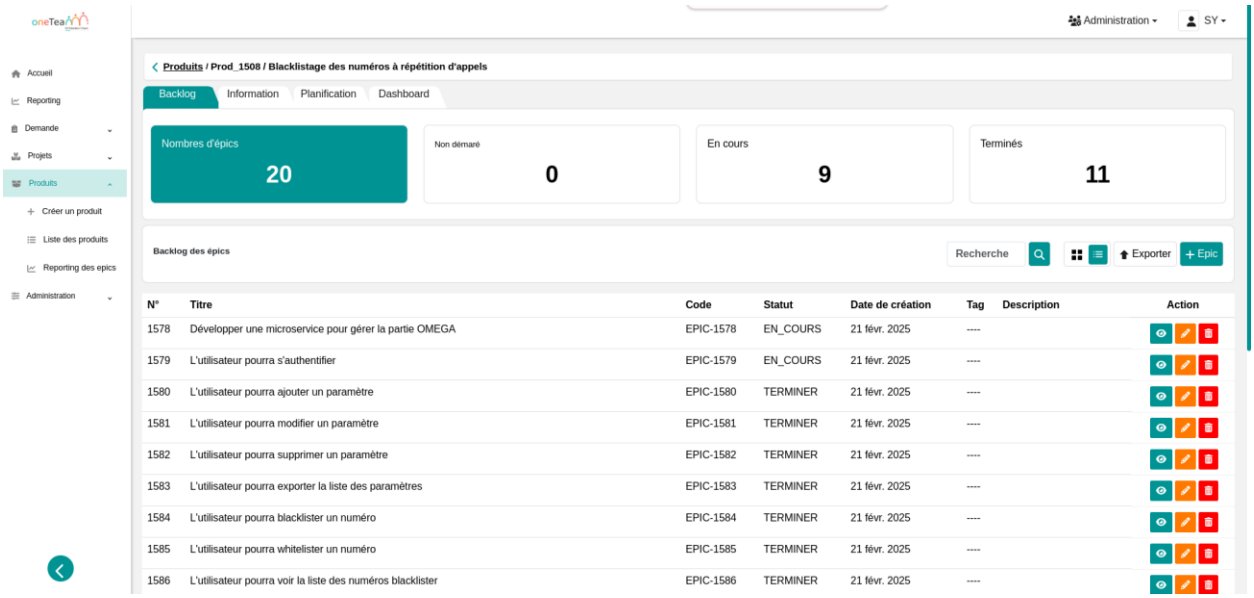

Figure 21: Présentation de GDD

V.2.3 Organisation des tableaux et des colonnes

Dans GDD, notre projet est structuré selon plusieurs niveaux hiérarchiques :

- **Produit** : "Blacklistage des numéros à répétition d'appels" (Prod_1508)
- **Épics** : Grandes fonctionnalités comme "Développer une microservice pour gérer la partie OMEGA" (1578)
- **Tâches** : Unités de travail spécifiques affectées aux membres de l'équipe

Les colonnes de notre tableau principal représentent les différents états de développement :

- **Non démarré** : Tâches planifiées mais pas encore commencées
- **En cours** : Tâches en cours de développement
- **Terminés** : Tâches complétées et validées

Cette organisation permet une visualisation rapide de l'état d'avancement global du projet, comme le montre la première capture d'écran où nous pouvons voir 0 tâches non démarrées, 9 en cours et 11 terminées.

V.2.4 Gestion des sprints et des tâches

Chaque sprint de deux semaines est planifié dans GDD avec un ensemble de tâches issues du backlog. Les épics sont décomposés en tâches plus petites et estimables, permettant une meilleure gestion du temps et des ressources.

La gestion des tâches comprend plusieurs étapes :

1. **Création et estimation** : Les tâches sont créées avec un titre descriptif, un code unique (comme EPIC-1578), une description détaillée et une estimation de l'effort requis.
2. **Assignation** : Chaque tâche est assignée à un membre de l'équipe responsable de sa réalisation.
3. **Suivi** : L'avancement est mis à jour régulièrement, permettant de visualiser la progression du sprint.
4. **Validation** : Une fois terminée, la tâche passe par un processus de validation avant d'être marquée comme "Terminée".

V.3 Procédures d'automatisation

L'automatisation des règles de gestion des appels répétitifs constitue l'élément clé de notre solution. Cette section détaille les procédures techniques implémentées pour assurer un fonctionnement autonome du système.

Modèle de données pour l'automatisation

Notre solution s'appuie sur un modèle de données optimisé pour la détection des comportements d'appels répétitifs. L'entité centrale CallHist stocke l'historique des appels avec les attributs suivants :

- chMsisdn : Numéro de téléphone de l'appelant
- chDate : Date et heure précise de l'appel
- chIdCall : Identifiant unique de l'appel

- chService : Service client concerné par l'appel

V.4 Présentation de l'application

La page de connexion

Le module de connexion représente le point d'entrée sécurisé de notre application. Développé durant le premier sprint, ce composant critique s'appuie sur Keycloak, une solution open-source robuste de gestion des identités et des accès. L'intégration de Keycloak offre plusieurs avantages significatifs :

- **Authentification centralisée** : Mise en œuvre d'un système Single Sign-On (SSO) permettant aux utilisateurs d'accéder à l'ensemble des applications Sonatel avec une seule authentification
- **Gestion fine des rôles et permissions** : Configuration de profils d'accès distincts selon les responsabilités des utilisateurs (administrateurs, opérateurs, superviseurs)
- **Protocoles standards** : Support natif des protocoles OAuth 2.0 et OpenID Connect garantissant une sécurité conforme aux standards actuels
- **Adaptabilité** : Possibilité d'intégrer des mécanismes d'authentification multi-facteurs selon l'évolution des besoins en sécurité

L'implémentation a nécessité la configuration d'un realm dédié dans Keycloak et la définition précise des flux d'authentification adaptés au contexte d'utilisation du service client.

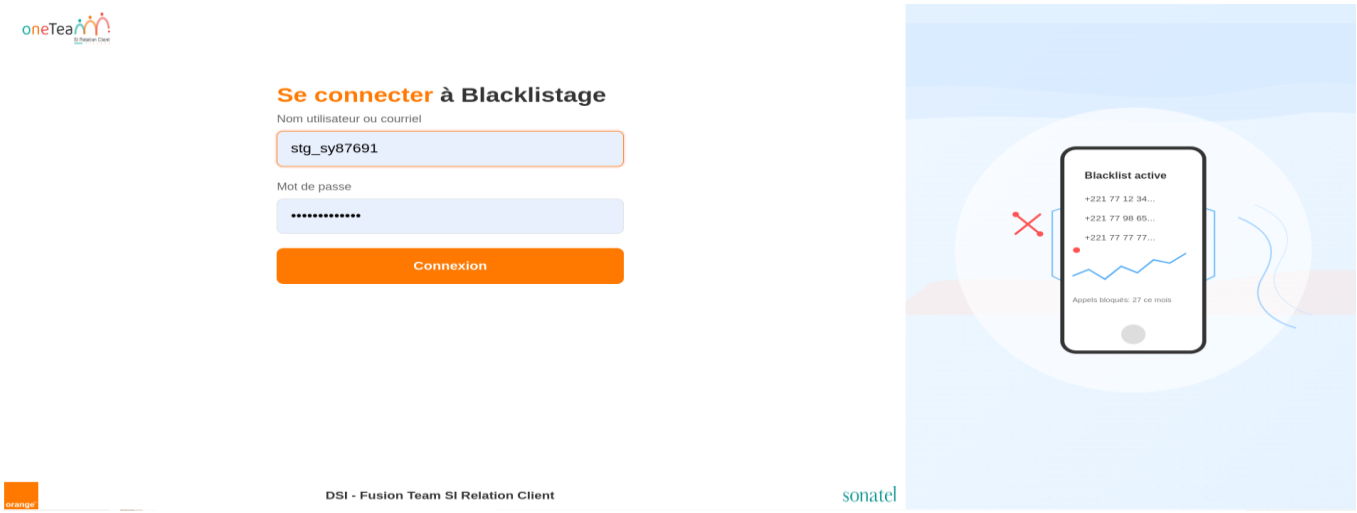

Figure 22: La page de connexion

La gestion des paramètres

Ce module configure le comportement du système selon les besoins opérationnels :

- Définition des seuils de déclenchement du blacklistage automatique
- Personnalisation des règles selon les services concernés
- Gestion des exceptions et cas particuliers
- Configuration des durées standard de restriction

Code backend

```java
@RestController
@RequestMapping("/api/parametres")
public class ParametreResource {

    private static final Logger LOG = LoggerFactory.getLogger(ParametreResource.class);

    private static final String ENTITY_NAME = "blacklistageOmegaBackendParametre";

    @Value("${jhipster.clientApp.name}")
    private String applicationName;

    private final ParametreService parametreService;

    private final ParametreRepository parametreRepository;

    private final ParametreQueryService parametreQueryService;

    public ParametreResource(
        ParametreService parametreService,
        ParametreRepository parametreRepository,
        ParametreQueryService parametreQueryService
    ) {
        this.parametreService = parametreService;
        this.parametreRepository = parametreRepository;
        this.parametreQueryService = parametreQueryService;
    }

    /**
     * {@code POST  /parametres} : Create a new parametre.
     *
     * @param parametreDTO the parametreDTO to create.
     * @return the {@link ResponseEntity} with status {@code 201 (Created)} and with body the new parametreDTO, or with status {@code 400 (Bad Request)} if the parametre has already an ID.
     * @throws URISyntaxException if the Location URI syntax is incorrect.
     */
    @PostMapping("")
    public ResponseEntity<ParametreDTO> createParametre(@Valid @RequestBody ParametreDTO parametreDTO) throws URISyntaxException {
        LOG.debug("REST request to save Parametre : {}", parametreDTO);
        if (parametreRepository.existsById(parametreDTO.getParService())) {
            throw new BadRequestAlertException("parametre already exists", ENTITY_NAME, "idexists");
        }
        parametreDTO = parametreService.save(parametreDTO);
        return ResponseEntity.created(new URI("/api/parametres/" + parametreDTO.getParService()))
            .headers(HeaderUtil.createEntityCreationAlert(applicationName, false, ENTITY_NAME, parametreDTO.getParService()))
            .body(parametreDTO);
    }
```

Figure 23: Backend Paramètre

Code frontend

Composant

```
export class ParametreComponent implements OnInit {
  entries: IParametre[] = []; // Liste des paramètres
  services: { id: string; name: string }[] = [
    { id: '1413', name: 'Service dérangement ADSL' },
    { id: '1441', name: 'Assistance client prépayé' },
    { id: '1443', name: 'Assistance client postpaid' },
    { id: '1444', name: 'Assistance client Omoney' },
    { id: '338392218', name: 'Standard_Sonatel' },
    { id: '800002040', name: 'ADV_OEnergie' },
    { id: '800005050', name: 'Orange_Energie' },
    { id: '800006060', name: 'Distributeurs' },
    { id: '800800800', name: 'Hotline DDE' },
    { id: '338793535', name: 'Hotline CDS' },
    { id: '800810820', name: 'Fibre Optique' },
  ];
  showModal = false;
  isEditMode = false;
  currentIndex: number | null = null;
  currentEntry: IParametre = { parService: '', parSeuil: 0, parDuree: 0, parPeriod: 0 };

  constructor(
    private parametreService: ParametreService,
    public router: Router, // ✅ Ajout du Router
  ) {}

  ngOnInit(): void {
    this.loadParameters();
  }

  trackParService(index: number, item: IParametre): string {
    return item.parService;
  }

  navigateToWithComponentValues(params: { predicate: string; order: string; ascending: boolean }): void {
    this.router.navigate([], {
      queryParams: {
        predicate: params.predicate,
        order: params.order,
        ascending: params.ascending,
      },
    });
  }

```

Figure 24: Composant Paramètre

Html

```html
<div class="card table-card">
  <div class="table-header">
    <h2 class="card-title">
      <i class="bi bi-gear"></i>
      Gestion des Paramètres
    </h2>

    <div class="table-actions">
      <button class="add-button" (click)="openModal()">
        <i class="bi bi-plus-lg"></i>
        Ajouter
      </button>
      <button type="button" style="padding: 0 0.75rem; height: 38px !important" class="btn btn-outline" (click)="onExport()">
        <i class="bi bi-download"></i>
        Exporter
      </button>
    </div>
  </div>

  <div class="table-responsive">
    <table class="data-table">
      <thead>
        <tr>
          <th>Service</th>
          <th>Seuil</th>
          <th>Durée</th>
          <th>Période</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        <tr *ngFor="let entry of entries; let i = index" class="table-row">
          <td>
            <span class="badge">{{ entry.parService }}</span>
          </td>
          <td>{{ entry.parSeuil }}</td>
          <td>{{ entry.parDuree }}</td>
          <td>{{ entry.parPeriod }}</td>
          <td class="d-flex justify-content-end gap-1">
            <button class="btn btn-primary" (click)="openModal(true, i)">
              <i class="bi bi-pencil"></i>
              Modifier
            </button>
            <button class="btn btn-dark" (click)="deleteEntry(i)">
              <i class="bi bi-trash"></i>
              Supprimer
            </button>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</div>
```

Figure 25: html de paramètre

Résultat

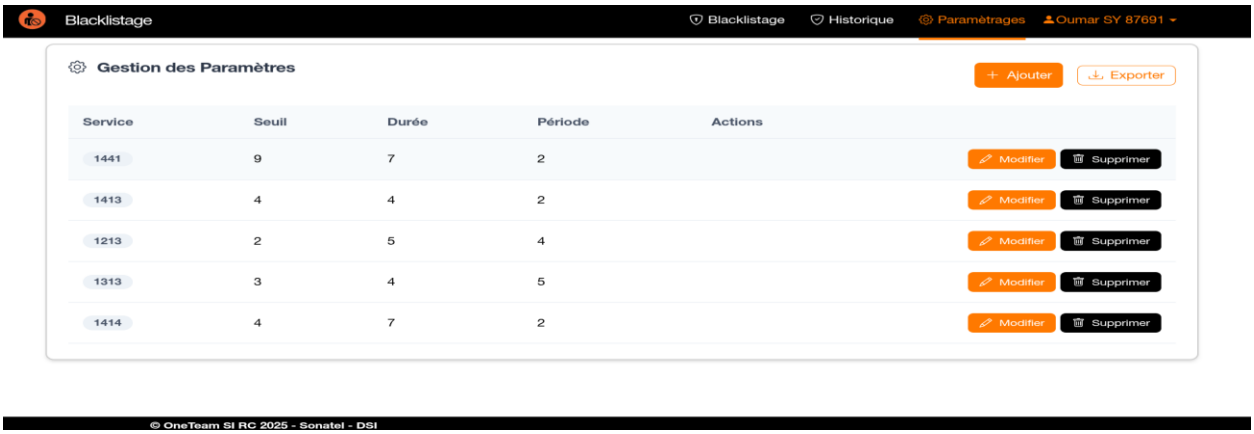

Figure 26: paramètre

Le module de blackliste

Ce module central permet la gestion efficace des numéros à appels répétitifs avec :

- Des fonctionnalités de recherche par MSISDN et critères multiples
- Un système de blacklistage temporisé avec dates précises
- Des outils d'analyse statistique des comportements d'appel
- Une interface optimisée pour les opérations quotidiennes du service client

Code backend

```java
/**
 * REST controller for managing {@link sn.sonatel.dsi.ftsirc.domain.Blacklst}.
 */
@RestController
@RequestMapping("/api/blacklsts")
public class BlacklstResource {

    private static final Logger LOG = LoggerFactory.getLogger(BlacklstResource.class);

    private static final String ENTITY_NAME = "blacklistageOmegaBackendBlacklst";

    @Value("${jhipster.clientApp.name}")
    private String applicationName;

    private final BlacklstService blacklstService;

    private final BlacklstRepository blacklstRepository;

    private final BlacklstQueryService blacklstQueryService;

    public BlacklstResource(
        BlacklstService blacklstService,
        BlacklstRepository blacklstRepository,
        BlacklstQueryService blacklstQueryService
    ) {
        this.blacklstService = blacklstService;
        this.blacklstRepository = blacklstRepository;
        this.blacklstQueryService = blacklstQueryService;
    }

    /**
     * {@code POST  /blacklsts} : Create a new blacklst.
     *
     * @param blacklstDTO the blacklstDTO to create.
     * @return the {@link ResponseEntity} with status {@code 201 (Created)} and with body the new blacklstDTO, or with status {@code 400 (Bad Request)} if the blacklst has already an ID.
     * @throws URISyntaxException if the Location URI syntax is incorrect.
     */
    @PostMapping("")
    public ResponseEntity<BlacklstDTO> createBlacklst(@Valid @RequestBody BlacklstDTO blacklstDTO) throws URISyntaxException {
        LOG.debug("REST request to save Blacklst : {}", blacklstDTO);
        if (blacklstDTO.getId() != null) {
            throw new BadRequestAlertException("A new blacklst cannot already have an ID", ENTITY_NAME, "idexists");
        }
        blacklstDTO = blacklstService.save(blacklstDTO);
        return ResponseEntity.created(new URI("/api/blacklsts/" + blacklstDTO.getId()))
            .headers(HeaderUtil.createEntityCreationAlert(applicationName, false, ENTITY_NAME, blacklstDTO.getId().toString()))
            .body(blacklstDTO);
    }
```

Code frontend

Composant

```
@Component({
  selector: 'jh1-blacklst',
  templateUrl: './blacklst.component.html',
  styleUrl: 'blacklst.component.scss',
  imports: [RouterModule, FormsModule, SharedModule, SortDirective, SortByDirective, FormatMediumDatetimePipe, ReactiveFormsModule],
})
export class BlacklstComponent implements OnInit {
  subscription: Subscription | null = null;
  blacklsts = signal<IBlacklst[]>([]);
  displayedBlacklsts = signal<IBlacklst[]>([]); // Signal pour les données paginées à afficher
  parametres = signal<IParametre[]>([]);
  isLoading = false;
  addForm: FormGroup;
  searchForm: FormGroup;
  Math = Math;
  isSearchVisible = false;
  isAllSelected = false;
  selectedFile: File | null = null;
  isUploading = false;
  resultMessage = '';
  hasError = false;
  services: { id: string; name: string }[] = [
    { id: '1413', name: 'Service dérangement ADSL' },
    { id: '1441', name: 'Assistance client prépayé' },
    { id: '1443', name: 'Assistance client postpaid' },
    { id: '1444', name: 'Assistance client Omoney' },
    { id: '338392218', name: 'Standard_Sonatel' },
    { id: '800002040', name: 'ADV_OEnergie' },
    { id: '800005050', name: 'Orange_Energie' },
    { id: '800000000', name: 'Distributeurs' },
    { id: '800800800', name: 'Hotline DDE' },
    { id: '338793535', name: 'Hotline CDS' },
    { id: '800810820', name: 'Fibre Optique' },
  ];

  sortState = sortStateSignal({ predicate: 'id', ascending: true });
  filters: IFilterOptions = new FilterOptions();

  itemsPerPage = ITEMS_PER_PAGE;
  page = 1;
  totalItems = 0;
  totalPages = 1;

  public readonly router = inject(Router);
  originalBlacklsts: IBlacklst[] = [];
  protected readonly blacklstService = inject(BlacklstService);
  protected readonly parametreService = inject(ParametreService);
  protected readonly historiqueService = inject(HistoriqueService);
  protected readonly activatedRoute = inject(ActivatedRoute);
  protected readonly sortService = inject(SortService);
  protected modalService = inject(NgbModal);
  protected ngZone = inject(NgZone);
```

Html

```
<div class="inner-container">
  <div class="card table-card">
    <div class="table-header">
      <h2 class="card-title">
        <i class="bi bi-list"></i>
        Liste des numéros blacklistés
      </h2>

      <div class="table-actions">
        <button class="search-toggle" [class.active]="isSearchVisible" (click)="toggleSearchPanel()">
          <i class="bi bi-search"></i>
          Recherche avancée
          <i class="bi bi-chevron-down"></i>
        </button>

        <button class="btn btn-success" [disabled]="!hasSelection()" (click)="onWhitelistSelected()">
          <i class="bi bi-shield-check"></i>
          Whitelister la sélection
        </button>
        <button type="button" class="btn btn-outline" (click)="exportData()">
          <i class="bi bi-download"></i>
          Exporter
        </button>
      </div>
    </div>

    <!-- Panneau de recherche masqué par défaut -->
    <div class="search-panel mb-4" [class.visible]="isSearchVisible">
      <form [formGroup]="searchForm" (submit)="applyFilters()" class="search-form">
        <div class="search-row">
          <div class="search-input-group">
            <label class="search-input">Date Debut</label>
            <input type="date" formControlName="dateDebut" class="add-input-field" placeholder="Date début" />
          </div>

          <div class="search-input-group">
            <label class="search-input">Date Fin</label>
            <input type="date" formControlName="dateFin" class="add-input-field" placeholder="Date fin" />
          </div>

          <div class="search-input-group">
            <label class="search-input">Téléphone</label>
            <input type="text" formControlName="searchNumero" class="add-input-field" placeholder="Rechercher un numéro" />
          </div>

          <div class="search-input-group">
            <label class="search-input">Service</label>
            <select formControlName="searchService" class="add-input-field">
              <option value="NOLIMIT">Tous les services</option>
              @for (param of services; track param.id) {
                <option [value]="param.id">{{ param.id }}</option>
              }
            </select>
          </div>

          <button type="submit" class="search-button">
            <i class="bi bi-search"></i>
            Rechercher
          </button>

          <button type="button" class="search-button-reset" (click)="resetFilters()">
            <i class="bi bi-x"></i>
            Réinitialiser
          </button>
        </div>
      </form>
    </div>
```

Figure 27: html blackliste

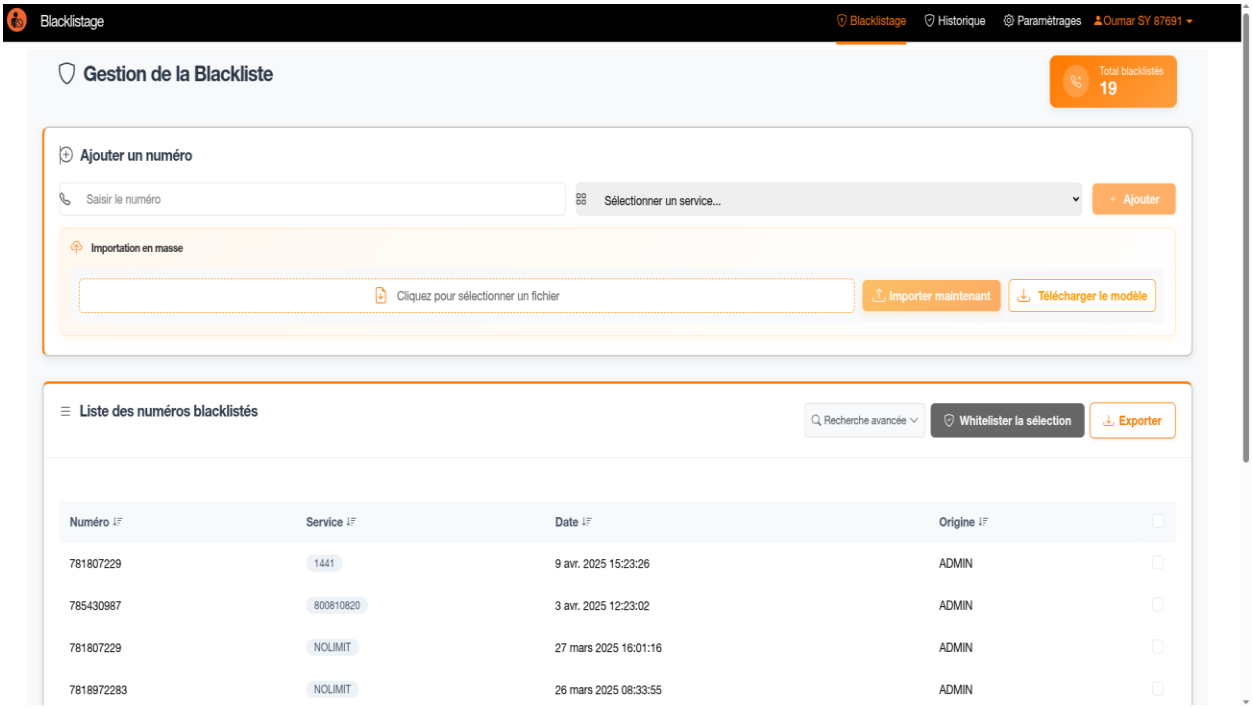

Figure 28: page Blacklistage

Le module historique

L'historique assure la traçabilité complète avec :

- L'enregistrement détaillé de chaque opération de blacklistage/whitelistage
- L'identification des opérateurs responsables des actions
- Des outils d'analyse temporelle et statistique
- Des fonctionnalités d'exportation pour les rapports périodiques

Code backend

```java
/**
 * REST controller for managing {@link sn.sonatel.dsi.ftsirc.domain.Historique}.
 */
@RestController
@RequestMapping("/api/historiques")
public class HistoriqueResource {

    private static final Logger LOG = LoggerFactory.getLogger(HistoriqueResource.class);

    private static final String ENTITY_NAME = "blacklistageBackendHistorique";

    @Value("${jhipster.clientApp.name}")
    private String applicationName;

    private final HistoriqueService historiqueService;

    private final HistoriqueRepository historiqueRepository;

    private final HistoriqueQueryService historiqueQueryService;

    public HistoriqueResource(
        HistoriqueService historiqueService,
        HistoriqueRepository historiqueRepository,
        HistoriqueQueryService historiqueQueryService
    ) {
        this.historiqueService = historiqueService;
        this.historiqueRepository = historiqueRepository;
        this.historiqueQueryService = historiqueQueryService;
    }

    /**
     * {@code POST  /historiques} : Create a new historique.
     *
     * @param historiqueDTO the historiqueDTO to create.
     * @return the {@link ResponseEntity} with status {@code 201 (Created)} and with body the new historiqueDTO, or with status {@code 400 (Bad Request)} if the historique has already an ID.
     * @throws URISyntaxException if the Location URI syntax is incorrect.
     */
    @PostMapping("")
    public ResponseEntity<HistoriqueDTO> createHistorique(@RequestBody HistoriqueDTO historiqueDTO) throws URISyntaxException {
        LOG.debug("REST request to save Historique : {}", historiqueDTO);
        if (historiqueDTO.getId() != null) {
            throw new BadRequestAlertException("A new historique cannot already have an ID", ENTITY_NAME, "idexists");
        }
        historiqueDTO = historiqueService.save(historiqueDTO);
        return ResponseEntity.created(new URI("/api/historiques/" + historiqueDTO.getId()))
            .headers(HeaderUtil.createEntityCreationAlert(applicationName, false, ENTITY_NAME, historiqueDTO.getId().toString()))
            .body(historiqueDTO);
    }
```

Figure 29: Backend historique

Code frontend

Le composant

```angular2
@Component({
  selector: 'jhi-historique',
  templateUrl: './historique.component.html',
  styleUrl: './historique.component.scss',
  imports: [RouterModule, FormsModule, SharedModule, FormatMediumDatetimePipe, ReactiveFormsModule],
})
export class HistoriqueComponent implements OnInit {
  public readonly router = inject(Router);
  searchForm: FormGroup | undefined;

  subscription: Subscription | null = null;
  historiques = signal<IHistorique[]>([]);
  filteredHistoriques = signal<IHistorique[]>([]);
  isLoading = false;

  sortState = sortStateSignal({ predicate: 'id', ascending: true });
  filters: IFilterOptions = new FilterOptions();
  itemsPerPage = ITEMS_PER_PAGE;
  totalItems = 0;
  page = 1;

  protected readonly formBuilder = inject(FormBuilder);
  protected readonly Math = Math;
  protected readonly historiqueService = inject(HistoriqueService);
  protected readonly activatedRoute = inject(ActivatedRoute);
  protected readonly sortService = inject(SortService);
  protected modalService = inject(NgbModal);
  protected ngZone = inject(NgZone);

  trackId = (item: IHistorique): number => this.historiqueService.getHistoriqueIdentifier(item);

  ngOnInit(): void {
    this.load();
    this.createSearchForm();

    this.subscription = combineLatest([this.activatedRoute.queryParamMap, this.activatedRoute.data])
      .pipe(
        tap(([params, data]) => this.fillComponentAttributeFromRoute(params, data)),
        tap(() => this.load()),
      )
      .subscribe();

    this.filters.filterChanges.subscribe(filterOptions => this.handleNavigation(1, this.sortState(), filterOptions));
  }
```

Figure 30: le composant Historique

Html

```html
<div class="whitelist-container mt-4">
  <!-- En-tête -->
  <div class="header-section">
    <div class="header-content">
      <h1 class="main-title">
        <i class="bi bi-clock-history"></i>
        Historique des actions
      </h1>
    </div>
  </div>

  <!-- Section principale -->
  <div class="main-content">
    <!-- Recherche -->
    <div class="card search-card">
      <form class="search-form" (ngSubmit)="onSearch()">
        <div class="date-range">
          <div class="form-group">
            <label>Date Début</label>
            <input type="date" formControlName="debut" class="input-field" placeholder="YYYY-MM-DD" />
          </div>
          <div class="form-group">
            <label>Date Fin</label>
            <input type="date" formControlName="fin" class="input-field" placeholder="YYYY-MM-DD" />
          </div>
          <div class="form-group">
            <label>MSISDN</label>
            <input type="text" formControlName="msisdn" class="input-field" placeholder="Rechercher un numéro" />
          </div>
          <button type="submit" class="btn btn-primary" style="height: 48px; width: 135px; margin-top: 35px">
            <i class="bi bi-search"></i>
            Recherche
          </button>
          <button type="button" class="search-button-reset" (click)="resetFilters()">
            <i class="bi bi-x"></i>
            Réinitialiser
          </button>
        </div>
      </form>
    </div>

    <!-- Tableau des résultats -->
    <div class="card table-card">
      <div class="table-header">
        <h2 class="card-title">
          <i class="bi bi-list-ul"></i>
          Liste des historiques
        </h2>
        <div class="table-actions">
          <button type="button" class="btn btn-outline" (click)="exportData()">
```

Figure 31: html d'Historique

Le résultat

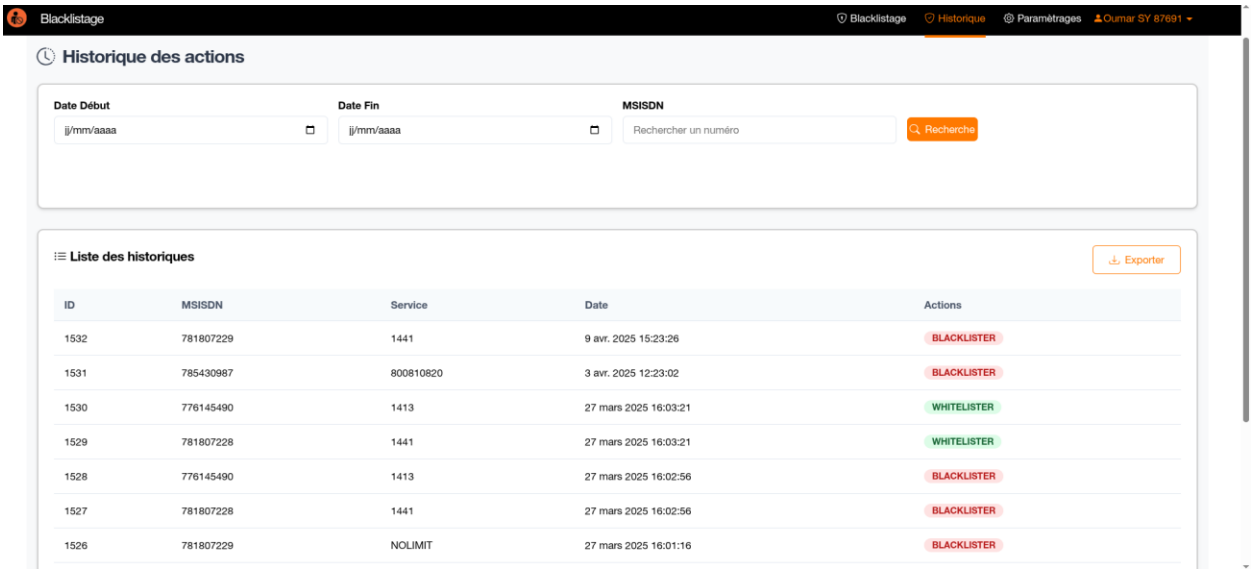

Figure 32: la page historique

Le Dashboard

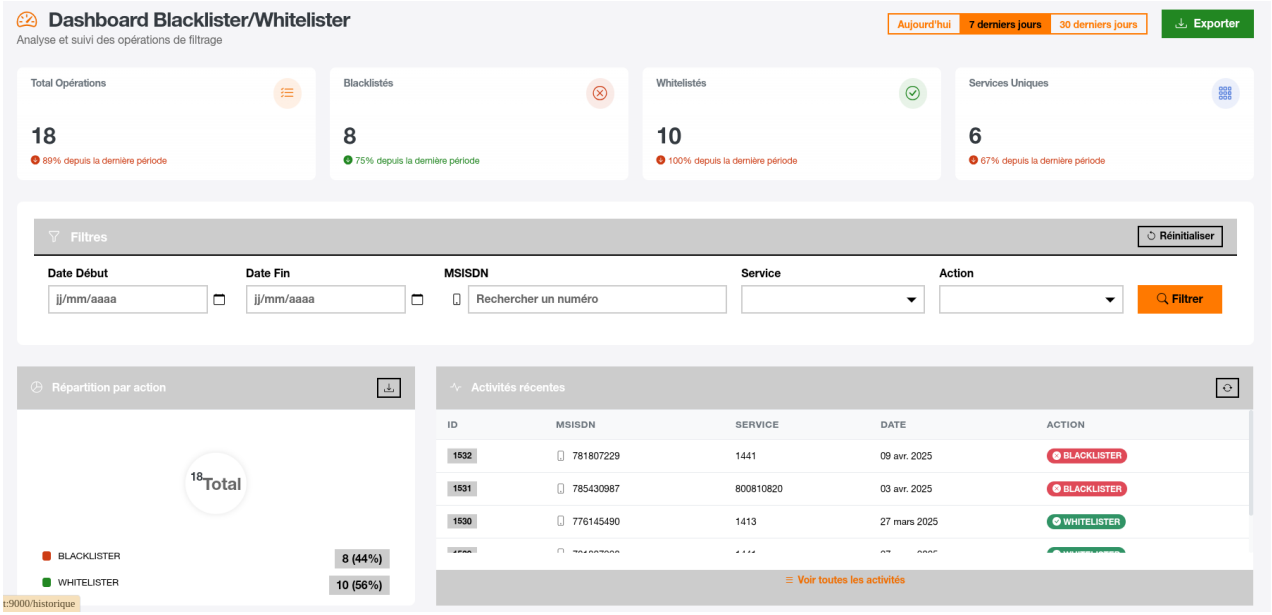

Figure 33: notre Dashboard

L'application est déployée en iso-prod sur le lien : https://blacklistage-dev.apps.malaaw-rec.orange-sonatel.com/ qui n'est accessible qu'en interne de la Sonatel.

Conclusion Générale

Ce projet de stage, réalisé dans le cadre de la gestion des appels vers le service client de SONATEL, a permis de répondre à une problématique majeure induite par l'application du décret relatif à la gratuité des appels. Ce décret, bien que bénéfique pour les clients légitimes en facilitant leur accès au service d'assistance, a également engendré des effets secondaires non négligeables, notamment une recrudescence des appels plaisantins et répétitifs. Ces comportements ont entraîné une saturation des plateaux téléphoniques, compromettant ainsi la qualité de service offerte aux utilisateurs ayant des besoins réels.

La solution développée au cours de ce stage repose sur l'automatisation des processus de « Blacklistage » et de « Whitelistage », permettant une gestion intelligente et différenciée des numéros clients. Grâce à cette approche, il est désormais possible d'identifier les appelants effectuant des appels excessifs et de les orienter vers un parcours adapté, tout en garantissant un accès fluide aux clients légitimes. Les mécanismes mis en place, basés sur des critères spécifiques tels que le nombre d'appels autorisés, la période d'observation et la durée de blocage, assurent une optimisation significative des ressources des plateaux téléphoniques.

Les étapes de conception, de réalisation et de déploiement de cette solution ont permis de démontrer son efficacité dans la réduction des appels non productifs et l'amélioration de l'expérience client. En automatisant la détection des comportements anormaux et en appliquant des stratégies de filtrage adaptées, cette solution contribue directement à la fluidification des flux d'appels et à la préservation de la qualité de service tant recherchée par SONATEL.

En conclusion, ce projet s'inscrit dans une démarche proactive visant à adapter les outils technologiques aux défis opérationnels rencontrés par les services clients. Il illustre également l'importance de l'innovation et de l'automatisation dans la résolution des problématiques modernes liées à la gestion des interactions clients. Les résultats obtenus ouvrent des perspectives intéressantes pour l'avenir, notamment en termes d'évolution des fonctionnalités existantes et de généralisation de cette solution à d'autres contextes similaires. Ce stage aura donc été une expérience enrichissante, tant sur le plan technique que sur celui de la compréhension des enjeux stratégiques et humains dans le secteur des télécommunications.

Webographie

Documentation technique

- JHipster: https://www.jhipster.tech/microservices-architecture/
- Angular : https://angular.io/docs
- Spring Boot: https://spring.io/projects/spring-boot
- Java : https://dev.java/learn/

Articles et tutoriels

- "Créer une application microservices avec JHipster": https://www.baeldung.com/jhipster-microservices
- "Angular pour les applications télécom": https://blog.angular.io/tagged/case-study
- "Gestion des appels indésirables - ARCEP": https://www.arcep.fr/la-regulation/grands-dossiers-thematiques-transverses/lutte-contre-les-appels-indesirables.html

Références sectorielles

- SONATEL/Orange Sénégal: https://www.orange.sn/
- ARTP Sénégal: https://www.artpsenegal.net/

Frameworks et bibliothèques

- PrimeNG (composants UI pour Angular): https://primeng.org/
- Spring Cloud (pour microservices): https://spring.io/projects/spring-cloud

Forums techniques

- Stack Overflow - Tags JHipster: https://stackoverflow.com/questions/tagged/jhipster
- GitHub JHipster: https://github.com/jhipster/generator-jhipster

Table des matières

Dédicace i
Remerciements ii
Table des Figures iii
Table des Tableaux iv
Abstract vi
Glossaire vii
INTRODUCTION ***1***
I. Présentation de l'organisme d'accueil 2
I.1. Présentation de la société 2
I.2. Présentation de l'équipe et du cadre de travail 3
II. Présentation du sujet et des objectifs 4
6.1. Les outils de travail 7
5.2. Langages de programmation 10
5.3. Outils déploiement 11
5.4 Étude comparative des frameworks existants 13
5.5 Sécurité et authentification 14
III. La méthodologie 15
2. Les méthodes agiles 18
3. Comparaison entre approche classique et approche agile 27
IV. Conception détaillées 30
IV.1. Les diagrammes des cas d'utilisation. 30
IV.2. Le diagramme de classe 38
IV.2.3. Diagramme de classe de l'application 39
IV.3. Les diagrammes des Séquence 39
IV.4 Prototypage de la solution 43
V. Le développement de l'application 47
V.1 Méthodologie de gestion de projet 47
V.2 Utilisation de l'application GDD pour la gestion de projet 47
V.4 Présentation de l'application 50
Conclusion Générale 60
Webographie 61